FastMatMul: A Code-Based Matrix-Multiplication Checking Layer for Verifiable Computation

Anonymous Submission

***Abstract*—Verifiable computation systems frequently need to prove large matrix multiplication statements, but naïve SNARK arithmetization of a $k \times k$ product costs $\mathcal{O}(k^3)$ constraints and even Freivalds' classical reduction leaves $\mathcal{O}(k^2)$ circuit work. We present FastMatMul, a matrix-multiplication checking layer that reduces the SNARK relation to $\mathcal{O}(tk + E_{\mathsf{msg}}(k))$ constraints plus $\mathcal{O}(t \log n + E_{\mathsf{link}}(k))$ auxiliary opening/linking work, where t is a security-parameter-sized constant. The protocol combines Freivalds' randomized check with proximity testing over row-wise encoded matrix commitments; a sampled-opening backend fixes intermediate vectors before sampling and links the circuit's sampled values to small commitments authenticated by Merkle paths.**

We prove perfect completeness and computational soundness assuming that another layer has already certified the encoded input commitments, and assuming the soundness of the intermediate binding/linking backend, with error $\epsilon_{\mathsf{SNARK}} + \epsilon_{\mathsf{cc}} + \epsilon_{\mathsf{bind}} + (k-1)/|\mathbb{F}| + 4(1-\delta)^t$. The Fiat–Shamir variant adds the standard random-oracle loss $\epsilon_{\mathsf{FS}}(q_H)$. The checker is modular: it verifies multiplication over these certified commitments, while the surrounding system certifies that they were derived from the raw input matrices.

We implement the checking layer in Rust and compare it with a direct Freivalds SNARK baseline. At $k = 4{,}096$, $\mathsf{FastMatMul}_{\mathsf{QA}}$ gives a $30.3\times$ constraint reduction, a $7.24\times$ proving-time speedup, and a 147 KB online proof excluding setup/key/CRS material; it continues to $k = 8{,}192$ with a 168 KB online proof under the same convention. A GPT-2 checking-layer case study with 36 claims over certified commitments shows a $2.30\times$ constraint reduction and a $1.78\times$ proving-time speedup.

***Index Terms*—verifiable computation, matrix multiplication, SNARKs, code commitments, proximity testing**

1. Introduction

Large matrix multiplication is a recurring bottleneck in verifiable computation. A prover may wish to convince a verifier that committed matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ satisfy $\mathbf{AB} = \mathbf{C}$ without forcing the verifier, or an underlying SNARK circuit, to recompute the full product. This problem appears across outsourced linear algebra, proof-carrying data pipelines, and verifiable inference for modern neural networks, where projection and attention layers routinely contain matrix products with dimensions $k \geq 1{,}000$. Naïve SNARK arithmetization of a single $k \times k$ matrix multiplication generates $\mathcal{O}(k^3)$ arithmetic constraints, a cost that is impractical even for moderate k.

The $\mathcal{O}(k^2)$ barrier. Freivalds' classical randomized check [1] reduces the problem from $\mathcal{O}(k^3)$ to $\mathcal{O}(k^2)$: given matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ and a random vector $\mathbf{r}$, one verifies $\mathbf{rAB} = \mathbf{rC}$ instead of recomputing the full product. Several recent systems, including verifiable ML pipelines such as Mystique [2], zkML [3], and the Freivalds-based baseline in [4], adopt this strategy, encoding the vector-matrix products directly as SNARK constraints. The resulting $\mathcal{O}(k^2)$ circuit is an improvement, yet it remains expensive: in our optimized baseline, Freivalds already requires 50.5 million R1CS constraints and 405.04 s of proving time at $k = 4{,}096$. Thus the gap between the $\mathcal{O}(k^2)$ SNARK cost and practical large-matrix verification remains wide.

Our insight: from computing to checking. We observe that the $\mathcal{O}(k^2)$ barrier is *not* inherent to the verification problem—it arises because existing approaches *compute* the vector-matrix products inside the SNARK circuit. Our key idea is to shift the paradigm from "computing in the circuit" to "*checking* in the circuit": the prover performs the expensive matrix arithmetic *outside* the SNARK and commits to all intermediate results; the verifier then checks a small number of lightweight algebraic relations that suffice to detect any inconsistency.

The technical engine behind this shift is *proximity testing over linear error-correcting codes*. By encoding each matrix row-wise with a systematic linear code $\mathcal{C}$ of relative distance δ, the vector-matrix product $\mathbf{x} = \mathbf{rA}$ can be verified *column by column* in the encoded domain: the linearity of $\mathcal{C}$ ensures that a single inner product ("fold") of one codeword column suffices to check one coordinate of the encoded product. If the prover cheats on any relation, the committed codewords disagree in at least a δ-fraction of positions, and a random query detects the inconsistency with probability $\geq \delta$. Crucially, the SNARK circuit encodes the *local algebraic checks*—fold computations, sampled RS evaluations, wire-handle openings, and message-opening checks at t queried positions—while Merkle path verification and sampled-linking consistency over compact leaf handles are checked by auxiliary verifiers bound to the same transcript. The resulting in-circuit checking size is $\mathcal{O}(tk + E_{\mathsf{msg}}(k))$, with $\mathcal{O}(t \log n + E_{\mathsf{link}}(k))$ auxiliary opening/linking work. It is *linear* in the matrix dimension when t is a small security-parameter-dependent constant and no optional full-codeword backend is used. This is a checking-layer result: the input

commitments and the intermediate message/linking backend are explicit components of the statement, and an end-to-end verifiable-computation system must instantiate the raw input certification path.

Contributions. We present FastMatMul, a matrix-multiplication checking layer for verifiable computation that realizes this approach. Our contributions are as follows.

- **A novel code-based verification protocol.** We design a three-round public-coin protocol that combines a structured variant of Freivalds' randomized reduction with proximity testing over linear codes and Merkle-tree commitments. In our sampled-opening backend, message commitments to $\mathbf{x}, \mathbf{y}, \mathbf{z}$ are fixed before the query set is derived, and the circuit opens public wire handles for the sampled values used in the fold checks; auxiliary opening/linking verifiers connect those handles to authenticated external leaves. The resulting online SNARK relation has $\mathcal{O}(tk)$ sampled algebraic work for fixed security parameter, a significant reduction from the $\mathcal{O}(k^2)$ of Freivalds-based approaches. The construction should not be read as an end-to-end commitment-generation system: certified input commitments to $\mathbf{A}, \mathbf{B}, \mathbf{C}$ remain a deployment interface. The protocol is made non-interactive via the Fiat–Shamir heuristic [5].
- **Rigorous security analysis.** We prove that FastMatMul achieves perfect completeness and computational soundness with error bounded by $\epsilon_{\mathsf{SNARK}} + \epsilon_{\mathsf{cc}} + \epsilon_{\mathsf{bind}} + (k-1)/|\mathbb{F}| + 4(1-\delta)^t$, where δ is the code's relative distance, $\epsilon_{\mathsf{SNARK}}$ is the SNARK soundness error, ϵ_{cc} is the message-binding and sampled handle-linking error, and ϵ_{bind} is the Merkle commitment's binding error. The Fiat–Shamir non-interactive variant adds the standard random-oracle loss $\epsilon_{\mathsf{FS}}(q_H)$. The theorem is stated relative to certified public input commitments to row-wise encoded matrices and to the sampled-opening backend, matching the committed-input viewpoint used in code-based proof systems such as Brakedown [6] and Orion [7]. We also give a concrete security-budget table that separates the reported scaling rows from a strict 128-bit deployment target. The analysis decomposes the adversary's strategy into SNARK failure, message/linking failure, commitment forgery, algebraic evasion, and proximity evasion, and shows each is negligible for standard parameter choices (Section 5).
- **Concrete efficiency gains.** We implement the multiplication-checking layer of FastMatMul in Rust and evaluate it against a direct Freivalds SNARK baseline across matrix dimensions up to $k = 2^{13}$ with the sampled-opening backend. Key findings (Section 6):
  - *Constraint reduction.* The implemented checking-layer constraints grow as $\mathcal{O}(k)$ for fixed t, versus $\mathcal{O}(k^2)$ for Freivalds. The gap reaches $7.5\times$ at $k = 1{,}024$ (419,139 vs. 3,156,298 constraints) and $30.3\times$ at $k = 4{,}096$ (1,667,305 vs. 50,495,818 constraints); the checking-layer implementation also scales to $k = 8{,}192$ with 3,331,779 constraints.
  - *Proving-time speedup.* The current implementation $\mathsf{FastMatMul}_{\mathsf{QA}}$ outperforms Freivalds from $k = 256$ onward. At $k = 4{,}096$, prove times are $55.92\,\mathrm{s}$ for $\mathsf{FastMatMul}_{\mathsf{QA}}$ and $405.04\,\mathrm{s}$ for Freivalds, a $7.24\times$ speedup. Online proof bytes, excluding setup/key/CRS material, are $147\,\mathrm{KB}$ at $k = 4{,}096$ and $168\,\mathrm{KB}$ at $k = 8{,}192$.
  - *GPT-2 workload case study.* For a GPT-2 workload with 36 matrix-multiplication claims at sequence length 1,024, $\mathsf{FastMatMul}_{\mathsf{QA}}$ reduces constraints by $2.30\times$ and proving time by $1.78\times$ relative to Freivalds, while verification is $1.39\times$ slower because the current backend checks sampled opening and linking material.
  - *Negligible encoding overhead.* The prover's off-circuit encoding step takes less than $1\,\mathrm{s}$ even at $k = 2^{20}$, confirming that encoding does not introduce a new bottleneck.

Paper organization. Section 2 gives a high-level technical overview. Section 3 recalls the necessary background. Section 4 presents the formal construction and complexity analysis. Section 5 provides the security proof. Section 6 reports our implementation and evaluation. Section 7 concludes with related-work positioning, limitations, and future directions.

2. Technical Overview

We provide a high-level description of our protocol for verifiable matrix multiplication before presenting the formal construction in Section 4. The central challenge is to verify that a claimed product $\mathbf{C} = \mathbf{AB}$ for matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k\times k}$ is correct, while keeping the verifier's work substantially below the $\mathcal{O}(k^3)$ cost of recomputing the multiplication. Our approach combines three classical ideas—*randomized verification*, *error-correcting codes*, and *succinct commitment schemes*—in a way that decomposes the verification into components that are each efficient to check inside a SNARK circuit.

2.1. Starting Point: Freivalds' Algorithm and Its Limitations

The classical starting point for verifying matrix multiplication is Freivalds' algorithm [1]. Given matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k\times k}$ and a uniformly random vector $\mathbf{r} \in \mathbb{F}^k$, one checks whether $\mathbf{rAB} = \mathbf{rC}$. If $\mathbf{AB} \neq \mathbf{C}$, then $\mathbf{D} := \mathbf{AB} - \mathbf{C} \neq \mathbf{0}$, and the check $\mathbf{rD} = \mathbf{0}$ fails with probability at least $1 - k/|\mathbb{F}|$ by the Schwartz–Zippel lemma. This reduces a matrix-matrix verification to a vector-matrix computation, collapsing the problem from $\mathcal{O}(k^3)$ to $\mathcal{O}(k^2)$.

However, even the reduced $\mathcal{O}(k^2)$ cost is prohibitive when the check must be performed *inside a SNARK circuit*. In an R1CS or Plonk arithmetization, each field multiplication corresponds to a constraint, so encoding the vector-matrix products $\mathbf{rA}$, $\mathbf{rB}$, and $\mathbf{rC}$ directly would still produce $\mathcal{O}(k^2)$ constraints. For matrices arising in modern

deep-learning inference (where k can reach thousands), this SNARK circuit cost remains impractical.

2.2. From Vectors to Codewords: Leveraging Linear Codes

Our key observation is that the *linearity* of error-correcting codes can be exploited to push the bulk of the computation outside the circuit while retaining verifiability. Concretely, suppose we encode each row of a matrix $\mathbf{M} \in \mathbb{F}^{k\times k}$ using a systematic linear code $\mathcal{C} : \mathbb{F}^k \to \mathbb{F}^n$ with relative minimum distance δ. Write $\mathsf{Enc}(\mathbf{M})$ for the matrix whose i-th row is $\mathcal{C}(\mathbf{M}_{i,\star})$. Because $\mathcal{C}$ is $\mathbb{F}$-linear, taking a linear combination of codewords yields the codeword of the corresponding linear combination of messages:

$$\sum_i r_i \cdot \mathcal{C}(\mathbf{M}_{i,\star}) = \mathcal{C}\left(\sum_i r_i \cdot \mathbf{M}_{i,\star}\right). \tag{1}$$

In other words, the "fold" operation $\mathsf{Fold}(\mathsf{Enc}(\mathbf{M}), \mathbf{r}) := \sum_i r_i \cdot \mathsf{Enc}(\mathbf{M})_{i,\star}$ equals $\mathsf{Enc}(\mathbf{rM})$ exactly. This identity is the engine that allows us to *check vector-matrix products column-by-column in the encoded domain* without materializing the full vectors.

Commitment via Merkle trees. Each encoded matrix $\mathsf{Enc}(\mathbf{M})$ is committed column-by-column using a Merkle tree:

$$\mathsf{cm}_M \leftarrow \mathsf{M.CM}(\mathsf{Enc}(\mathbf{M})).$$

The verifier starts from certified public roots $(\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C)$ that bind the computation inputs and output at the cost of $\mathcal{O}(1)$ hash values.

2.3. Proximity-Based Verification via Random Sampling

After the Freivalds reduction, the verification task becomes: given committed encoded matrices $\mathsf{Enc}(\mathbf{A}), \mathsf{Enc}(\mathbf{B}), \mathsf{Enc}(\mathbf{C})$ and a random challenge r, check that the intermediate vectors

$$\mathbf{x} = \mathbf{rA}, \quad \mathbf{y} = \mathbf{xB}, \quad \mathbf{z} = \mathbf{rC}$$

satisfy $\mathbf{y} = \mathbf{z}$. We now explain how to verify these relations in *sub-linear* time using the encoding.

The prover computes $\mathbf{x}, \mathbf{y}, \mathbf{z}$ outside the SNARK, encodes them, commits to the message vectors and to their encoded leaves, and sends these commitments before the query tuple is sampled. The verifier then samples an ordered tuple $I = (j_1, \ldots, j_t) \leftarrow\!\$\ [n]^t$ of column indices independently with replacement. For each sampled $j \in I$, the prover supplies the j-th column or coordinate as Groth16 witness data, opens a compact public wire handle for that witness, and provides auxiliary opening/linking material tying the wire handle to the authenticated external leaf handle.

At each queried position j, the verifier checks four *local consistency equations*:

(i) $\mathsf{Enc}(\mathbf{x})_j = \mathsf{Fold}\big(\mathsf{Enc}(\mathbf{A})^{(j)}, \mathbf{r}\big)$, ensuring $\mathbf{x} = \mathbf{rA}$;
(ii) $\mathsf{Enc}(\mathbf{y})_j = \mathsf{Fold}\big(\mathsf{Enc}(\mathbf{B})^{(j)}, \mathbf{x}\big)$, ensuring $\mathbf{y} = \mathbf{xB}$;
(iii) $\mathsf{Enc}(\mathbf{z})_j = \mathsf{Fold}\big(\mathsf{Enc}(\mathbf{C})^{(j)}, \mathbf{r}\big)$, ensuring $\mathbf{z} = \mathbf{rC}$;
(iv) $\mathsf{Enc}(\mathbf{y})_j = \mathsf{Enc}(\mathbf{z})_j$, ensuring the Freivalds check $\mathbf{y} = \mathbf{z}$.

The crucial point is that each of these checks involves only a *single column* of the encoded matrices—an inner product of length k (for the fold operations) plus a scalar comparison. Hence the total verifier work per query is $\mathcal{O}(k)$, and for t queries the total is $\mathcal{O}(tk)$.

Why does sampling suffice?. The intermediate vectors are fixed before the query tuple is sampled. At each sampled position, the SNARK relation recomputes $\mathsf{Enc}(\mathbf{x})_j$, $\mathsf{Enc}(\mathbf{y})_j$, and $\mathsf{Enc}(\mathbf{z})_j$ from the committed message vectors and checks that these values match the opened encoded leaves. The verifier does not encode the full vectors; it verifies the SNARK proof and the sampled opening/linking material. If any of the underlying vector relations fails—say $\mathbf{y} \neq \mathbf{z}$—then the codewords $\mathsf{Enc}(\mathbf{y})$ and $\mathsf{Enc}(\mathbf{z})$ are distinct. By the minimum-distance property of $\mathcal{C}$, they disagree in at least a δ-fraction of positions. A uniformly random query lands on a disagreement with probability $\geq \delta$, so t independent queries detect the inconsistency except with probability at most $(1-\delta)^t$. By a union bound over the four checked relations, the overall proximity-check failure probability is at most $4(1-\delta)^t$. Setting $t = \lceil(\lambda+2)/\log_2(1/(1-\delta))\rceil$ drives this union-bounded proximity term below $2^{-\lambda}$.

2.4. Putting It Together: The Full Protocol Pipeline

Combining the ingredients above, our protocol $\mathsf{FastMatMul}$ proceeds as follows (see Figure 1 for the formal description).

Public input. The verifier is given certified roots $(\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C)$. They bind to row-wise encodings of $\mathbf{A}, \mathbf{B}, \mathbf{C}$ under the linear code $\mathcal{C}$.

Challenge. The verifier samples a random field element $r \in \mathbb{F}$ and sends it to the prover. This induces the structured challenge vector $\mathbf{r} = (1, r, r^2, \ldots, r^{k-1})$.

Respond. The prover computes the intermediate vectors $\mathbf{x} = \mathbf{rA}$, $\mathbf{y} = \mathbf{xB}$, $\mathbf{z} = \mathbf{rC}$, commits to the message vectors, encodes them, and commits to the encoded leaves $(\mathsf{cm}_x, \mathsf{cm}_y, \mathsf{cm}_z)$.

Query. The verifier samples $I = (j_1, \ldots, j_t) \leftarrow\!\$\ [n]^t$ and sends the sampled indices to the prover. The prover supplies the corresponding sampled columns or vector leaves as Groth16 witness data, together with public wire handles and auxiliary Merkle openings for compact leaf handles.

SNARK verification. The verifier checks message-commitment openings for the intermediate vectors, wire-handle openings for sampled values, sampled RS evaluations for intermediate vectors, and the four local consistency equations at each queried index. These checks are arithmetized into the Groth16 relation defined in Section 4. Merkle path validity and sampled-linking consistency over the public wire/leaf handles are auxiliary verifier checks bound to the same transcript. The prover then sends a succinct proof π plus the sampled opening/linking material, and the verifier accepts only if every component verifies.

Cost summary. The *SNARK circuit size*—the key metric governing prover cost—is $\mathcal{O}(t \cdot k + E_{\mathsf{msg}}(k))$, where the first term includes sampled folds, RS evaluations, and wire-handle openings, and E_{msg} is the message-opening cost. Merkle authentication and sampled-linking verification add $\mathcal{O}(t \log n + E_{\mathsf{link}}(k))$ auxiliary online work. For the sampled-opening backend evaluated in this paper, the in-circuit checker is linear in k for fixed t and avoids the k^2 or k^3 circuit costs. The prover additionally performs the native matrix multiplication ($\mathcal{O}(k^3)$) and encoding ($\mathcal{O}(kn)$) outside the SNARK. The verifier's core SNARK work is $\mathcal{O}(1)$ for pairing-based backends, or $\mathcal{O}(|\mathsf{x}|)$ for transparent ones; total verification additionally depends on the selected commitment/linking backend. A detailed cost breakdown appears in Table 3 (Section 4.3).

3. Preliminaries

We introduce notation and recall the building blocks used in our construction. Throughout, $\mathbb{F}$ denotes a finite field of size $|\mathbb{F}| = p$ for a prime p, and λ denotes the security parameter. We write $\mathsf{negl}(\lambda)$ for any function that is negligible in λ, and PPT for probabilistic polynomial-time.

3.1. Notation

Matrices are denoted by bold uppercase letters $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$ and vectors by bold lowercase letters $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{F}^k$. For a matrix $\mathbf{M}$, we write $\mathbf{M}_{i,\star}$ for its i-th row and $\mathbf{M}_{\star,j}$ for its j-th column, both viewed as elements of $\mathbb{F}^k$. We write $[n] = \{1, 2, \ldots, n\}$ and $I \leftarrow\$ [n]^t$ to denote the uniform sampling of t indices from $[n]$ (with replacement). The expression $r \leftarrow\$ \mathbb{F}$ denotes uniform random sampling from $\mathbb{F}$.

3.2. Linear Error-Correcting Codes

A *linear code* over $\mathbb{F}$ is an $\mathbb{F}$-linear map $\mathcal{C} : \mathbb{F}^k \to \mathbb{F}^n$ defined by a generator matrix $G \in \mathbb{F}^{k \times n}$, so that $\mathcal{C}(\mathbf{m}) = \mathbf{m} \cdot G$ for any message $\mathbf{m} \in \mathbb{F}^k$. The code has *rate* $\rho = k/n$ and *minimum distance* $d = \min_{\mathbf{c} \neq \mathbf{c}'} \Delta(\mathbf{c}, \mathbf{c}')$ over distinct codewords $\mathbf{c}, \mathbf{c}' \in \mathcal{C}$, where Δ denotes the Hamming distance. The *relative distance* is $\delta = d/n$. A code is *systematic* if the first k coordinates of $\mathcal{C}(\mathbf{m})$ equal $\mathbf{m}$ itself.

We write $\mathsf{Enc}(\mathbf{m}) := \mathcal{C}(\mathbf{m})$ for the encoding of a message vector. For a matrix $\mathbf{M} \in \mathbb{F}^{k \times k}$, we define the *row-wise encoding* $\mathsf{Enc}(\mathbf{M}) \in \mathbb{F}^{k \times n}$ as the matrix whose i-th row is $\mathsf{Enc}(\mathbf{M}_{i,\star})$. The j-th column of $\mathsf{Enc}(\mathbf{M})$ is denoted $\mathsf{Enc}(\mathbf{M})^{(j)} \in \mathbb{F}^k$.

The key property we exploit is *linearity*: for any coefficients $r_0, \ldots, r_{k-1} \in \mathbb{F}$,

$$\sum_{i=0}^{k-1} r_i \cdot \mathsf{Enc}(\mathbf{M}_{i,\star}) = \mathsf{Enc}\left(\sum_{i=0}^{k-1} r_i \cdot \mathbf{M}_{i,\star} \right). \tag{2}$$

Definition 3.1 (Fold)**.** For vectors $\mathbf{c}, \mathbf{r} \in \mathbb{F}^k$ with $\mathbf{c} = (c_0, \ldots, c_{k-1})^\top$ and $\mathbf{r} = (r_0, \ldots, r_{k-1})$, define

$$\mathsf{Fold}(\mathbf{c}, \mathbf{r}) \coloneqq \langle \mathbf{r}, \mathbf{c} \rangle = \sum_{i=0}^{k-1} r_i \cdot c_i.$$

Equivalently, for the j-th column of a row-wise encoded matrix, $\mathsf{Fold}(\mathsf{Enc}(\mathbf{M})^{(j)}, \mathbf{r})$ equals the j-th coordinate of $\mathsf{Enc}(\mathbf{r}\mathbf{M})$. This follows directly from the linearity of the code (Equation 2).

Concrete instantiations. Our protocol is compatible with any linear code. Two natural choices are: (i) *Reed–Solomon codes*, where $\mathcal{C}(\mathbf{m})$ evaluates the polynomial $\sum_i m_i X^i$ over a domain of size $n > k$, achieving the optimal (MDS) distance $\delta = 1 - k/n + 1/n$; and (ii) *expander-based codes* as used in Brakedown [6], which achieve linear-time encoding $\mathcal{O}(n)$ with constant relative distance. We discuss the trade-offs in Section 4.3.

3.3. Merkle Tree Commitment

A *Merkle tree* [8] over a collision-resistant hash function $H : \{0,1\}^* \to \{0,1\}^\lambda$ provides a vector commitment scheme with the following algorithms:

- $\mathsf{cm} \leftarrow \mathsf{M.CM}(\mathbf{v})$: Given a vector $\mathbf{v} = (v_1, \ldots, v_n)$, construct a binary hash tree with leaves $H(v_1), \ldots, H(v_n)$ and output the root hash cm.
- $(\mathbf{v}_j, \pi_j) \leftarrow \mathsf{Merkle.Open}(\mathsf{cm}, j)$: Open position j, returning the leaf value v_j and the authentication path π_j (the sequence of sibling hashes from the leaf to the root).
- $b \leftarrow \mathsf{Merkle.Verify}(\pi_j, v_j, \mathsf{cm})$: Verify that v_j is the j-th leaf consistent with root cm; output $b \in \{0,1\}$.

The scheme is *position-binding*: under the collision resistance of H, no PPT adversary can produce (j, v, π) and (j, v', π') with $v \neq v'$ such that both verify against the same root cm, except with probability $\epsilon_{\mathsf{bind}}(\lambda) = \mathsf{negl}(\lambda)$.

3.4. Succinct Non-Interactive Arguments of Knowledge

A *SNARK* (Succinct Non-interactive Argument of Knowledge) for an NP relation $\mathcal{R}$ consists of algorithms $(\mathsf{Setup}, \mathsf{SNARK.P}, \mathsf{SNARK.V})$:

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R})$: Generate public parameters.
- $\pi \leftarrow \mathsf{SNARK.P}(\mathsf{pp}, \mathsf{x}, \mathsf{w})$: Given a statement x and witness w with $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}$, produce a proof π.
- $b \leftarrow \mathsf{SNARK.V}(\mathsf{pp}, \mathsf{x}, \pi)$: Verify the proof and output $b \in \{0,1\}$.

A SNARK satisfies *completeness* (an honest prover always convinces the verifier), *knowledge soundness* (any PPT prover that makes the verifier accept can be used to extract a valid witness), and *succinctness* (the proof size $|\pi|$ and verifier time are $\mathsf{poly}(\lambda, |\mathsf{x}|)$, independent of the witness size and computation).

Our protocol is modular with respect to the choice of SNARK backend, including pairing-based systems such as

Pinocchio [9] and Groth16 [10], as well as transparent or preprocessing systems such as Hyrax [11], Sonic [12], Marlin [13], and Spartan [14]. The key metric is the *circuit size* (number of R1CS constraints or PlonK gates) of the relation that the SNARK backend must prove, as this dominates the prover's cost.

3.5. Freivalds' Algorithm

Freivalds' algorithm [1] is a classical randomized algorithm for verifying matrix multiplication. Given $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k\times k}$, it samples $\mathbf{r} \leftarrow\$ \mathbb{F}^k$ and checks whether $\mathbf{r}(\mathbf{AB}) = \mathbf{rC}$. The correctness relies on the following:

Lemma 3.2 (Schwartz–Zippel [15], [16])**.** *Let $P(X_1, \ldots, X_m)$ be a non-zero polynomial of total degree d over a finite field $\mathbb{F}$. Then for $r_1, \ldots, r_m \leftarrow\$ \mathbb{F}$ chosen uniformly and independently,*

$$\Pr[P(r_1, \ldots, r_m) = 0] \leq \frac{d}{|\mathbb{F}|}.$$

In our protocol, we use a structured variant: instead of sampling $\mathbf{r} \leftarrow\$ \mathbb{F}^k$ uniformly, we sample one scalar $r \leftarrow\$ \mathbb{F}$ and set $\mathbf{r} = (1, r, r^2, \ldots, r^{k-1})$. This keeps the Freivalds challenge compact in the transcript and in the SNARK statement: the verifier records one field element rather than k independent ones. If $\mathbf{D} = \mathbf{AB} - \mathbf{C} \neq \mathbf{0}$, then for any non-zero column $\mathbf{d}_j$ of $\mathbf{D}$, the value $\langle \mathbf{r}, \mathbf{d}_j \rangle$ is a non-zero univariate polynomial in r of degree at most $k-1$. By Lemma 3.2, it vanishes with probability at most $(k-1)/|\mathbb{F}|$, which is negligible over the 256-bit fields used in our experiments.

4. Our Construction

In this section we present the formal construction of our verifiable matrix multiplication protocol FastMatMul. We begin by defining the relation chain that reduces the original matrix multiplication statement to a succinct SNARK-friendly relation (Section 4.1), then give the full protocol description (Section 4.2), and conclude with a complexity analysis (Section 4.3). Table 1 lists only the protocol-specific notation introduced in this section; basic field, matrix, code, Merkle, and SNARK notation is fixed in Section 3.

4.1. Relation Reduction Chain

Our construction proceeds through two successive reductions. Each reduction preserves completeness exactly and introduces a bounded soundness error that we analyze in Section 5.

Input-commitment model. Following the committed-oracle viewpoint common in code-based proof systems such as Brakedown [6] and Orion [7], we take the input matrix commitments as certified public inputs. Concretely, $\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C$ are assumed to bind to row-wise encodings $\mathsf{Enc}(\mathbf{A}), \mathsf{Enc}(\mathbf{B}), \mathsf{Enc}(\mathbf{C})$ of some matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k\times k}$. Certification means that every accepting opening of cm_M, for $M \in \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$, is a coordinate or column of $\mathsf{Enc}(M)$ for a fixed matrix M determined before the verifier samples the Freivalds challenge. This certification may be supplied by the surrounding committed-codeword system, by a public generation process, or by a separate input-opening proof. It is a precondition of the multiplication-checking protocol rather than an online cost charged to FastMatMul. The goal of FastMatMul is to verify the multiplication relation among these committed matrices without arithmetizing the full matrix product. The concrete deployment interface used for our claims is therefore: an offline certification layer publishes $(\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C)$ together with the assertion that they are binding commitments to row-wise encoded matrices; the online checker receives only these certified roots and proves the multiplication relation over them. If the deployment starts from raw matrices or ML tensors, the cost of producing and certifying those roots must be added separately. Throughout this section, $\mathsf{M.CM}(\mathsf{Enc}(\mathbf{M}))$ is shorthand for the Merkle root over compact leaf handles $(L_{M,1}, \ldots, L_{M,n})$, where each handle is binding to the corresponding encoded column $\mathsf{Enc}(\mathbf{M})^{(j)}$. For an encoded vector, $\mathsf{M.CM}(\mathsf{Enc}(v))$ analogously commits to handles for the length-n sequence of encoded coordinates. The sampled values themselves are opened only to the Groth16 relation as witness data; the auxiliary verifier sees handles and authentication paths.

Symbol	Description
$\mathcal{M}, \mathcal{W}$	Matrix objects $\{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$ and intermediate vectors $\{\mathbf{x}, \mathbf{y}, \mathbf{z}\}$
$\mathcal{S}$	Sampled-object set $\mathcal{M} \cup \mathcal{W}$
$I = (j_1, \ldots, j_t)$	Ordered tuple of sampled positions
cm_χ	Merkle root for committed object $\chi \in \mathcal{S}$
μ_v, ω_v	Message commitment to intermediate vector v and its opening data
$a_{\chi,j}$	Sampled object value used by the SNARK at position j
$L_{\chi,j}, \pi_{\chi,j}$	Authenticated leaf handle and opening path for object χ at position j
$\Gamma_{\chi,j}, \rho_{\chi,j}$	Public SNARK-side handle for $a_{\chi,j}$ and its opening randomness
σ_I	Auxiliary proof for sampled opening/linking checks
x, w	Public instance and private witness for the SNARK relation
$E_{\mathsf{msg}}(k), E_{\mathsf{link}}(k)$	Costs for message-opening and auxiliary linking checks

TABLE 1. PROTOCOL-SPECIFIC NOTATION USED IN SECTION 4.

Definition 4.1 (Sampled-opening backend)**.** The online checker uses a black-box backend for sampled openings. For each sampled object χ and query j, the Groth16 circuit uses a private value $a_{\chi,j}$ and publishes a handle $\Gamma_{\chi,j}$ to that value. The auxiliary verifier sees the committed root cm_χ, a leaf handle $L_{\chi,j}$, an opening path $\pi_{\chi,j}$, and an aggregate auxiliary proof σ_I. We write AuxV for the combined auxiliary verifier and LinkV for its sampled-linking check after path verification. The backend must provide three guarantees:

Circuit binding. If the Groth16 relation accepts, every sampled value used in the fold equations is bound to its public handle $\Gamma_{\chi,j}$.

Position opening. If the auxiliary verifier accepts, each

Contract item	What the main proof assumes
Input roots	$\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C$ bind to fixed row-wise codewords before r.
Intermediate commitments	μ_x, μ_y, μ_z and $\mathsf{cm}_x, \mathsf{cm}_y, \mathsf{cm}_z$ are fixed before I.
Circuit binding	Each sampled value used by Groth16 is bound to a public handle.
Opening/linking	Accepted auxiliary checks bind each handle to the opened leaf under the pre-query root.
Concrete backend	The artifact uses one batched linking proof with $\epsilon_{\mathsf{link}} \leq \epsilon_{\mathsf{ped}} + \epsilon_{\mathsf{QA}} + Q/\lvert\mathbb{F}\rvert$ for $Q = \lvert\mathcal{S}\rvert t$.

TABLE 2. BLACK-BOX BACKEND CONTRACT USED BY THE CONSTRUCTION AND SECURITY PROOF.

$L_{\chi,j}$ is authenticated as the leaf at position j under the pre-query root cm_χ.

Sampled linking. If both verifiers accept, the value bound to $\Gamma_{\chi,j}$ is the same value authenticated by $L_{\chi,j}$, except with probability $\epsilon_{\mathsf{link}}(\lambda)$ over the backend security experiment. The message commitments to the intermediate vectors have independent binding error ϵ_{msg}, so the theorem charges $\epsilon_{\mathsf{cc}} \leq \epsilon_{\mathsf{msg}} + \epsilon_{\mathsf{link}}$.

We do not require a new global code-commitment primitive. Instead, FastMatMul is stated relative to the committed-codeword viewpoint of Brakedown [6] and Orion [7], plus the sampled-opening interface above. The concrete artifact instantiates the interface with message commitments, Merkle openings, and a batched auxiliary linking proof; Appendix B gives the algebraic relation, batching argument, and online-byte accounting. The main construction and theorem use only the contract in Table 2.

Artifact backend in one paragraph. For the implementation evaluated in Section 6, each sampled SNARK-side handle and each authenticated leaf handle is a Pedersen-style algebraic handle to the same object. The auxiliary proof is one batched quasi-adaptive NIZK (QA-NIZK) proof for a linear relation [17] linking all $Q = |\mathcal{S}|t$ sampled handle pairs. Assuming binding of the handle parameters, soundness of the QA-NIZK for the linear batch relation, and a domain-separated batch challenge sampled after the batch statement is fixed, the linking error is $\epsilon_{\mathsf{link}} \leq \epsilon_{\mathsf{ped}} + \epsilon_{\mathsf{QA}} + Q/|\mathbb{F}|$. The CRS, handle generators, and proving/verifying keys are setup material; the reported byte counts in this paper are online proof bytes and exclude those serialized setup objects.

QA-NIZK, CRS, and generator model. The QA-NIZK is used only for the linear batch relation above, not for the matrix multiplication relation itself. Its CRS fixes the linear-relation verification parameters and the handle generator description used by the sampled-linking backend. Those generators may be serialized directly or expanded deterministically from a public seed, depending on the concrete deployment; either way, the corresponding bytes are setup/key/CRS material, not online proof bytes. The current artifact reports online proof material and setup time, but not the serialized size of these setup objects.

Backend boundary. The security theorem is compositional: it proves the multiplication-checking protocol relative to any backend satisfying Definition 4.1. An end-to-end system must choose one concrete input-certification interface and specify its contribution to preprocessing, public input size, and setup. Our implementation instantiates the intermediate-vector backend with message commitments, Merkle openings, and auxiliary sampled-linking checks; it does not certify raw input matrices by itself. For the GPT-2 case study, static model-weight roots can be certified once and reused, while per-request activation/output roots must be certified by the surrounding computation pipeline. Thus the GPT-2 experiment is a checking-layer case study over certified roots, not an end-to-end raw-tensor VC measurement.

Concrete deployment model for certified roots. The intended deployment is a two-layer pipeline. An offline or upstream certification layer encodes each raw matrix or tensor, forms the compact leaf handles, and publishes the Merkle root together with the claim that the root binds to the encoded object. The online multiplication checker then proves only the multiplication relation among these certified roots. For a $k \times k$ matrix with codeword length n, this certification layer performs $\Omega(kn)$ field work to produce the row-wise codeword and $\Omega(n)$ handle/hash work to form the root; a vector root costs $\Omega(n)$ plus the selected message-commitment cost. Static model-weight roots can amortize this cost over many inference requests. Per-request activation and output roots require an upstream tensor-commitment step; the measurements in Section 6 remain end-to-end beneficial only when the amortized certification cost is below the online checking-layer savings reported there.

The original relation $\mathcal{R}_{\mathsf{orig}}$. The starting point is the statement that the prover knows matrices whose *encoded* commitments satisfy the multiplication relation. The public instance is $(\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C)$ and the witness is $(\mathbf{A}, \mathbf{B}, \mathbf{C})$; the relation $\mathcal{R}_{\mathsf{orig}}$ requires:

$$\mathsf{cm}_A = \mathsf{M.CM}(\mathsf{Enc}(\mathbf{A})), \quad \mathsf{cm}_B = \mathsf{M.CM}(\mathsf{Enc}(\mathbf{B})),$$
$$\mathsf{cm}_C = \mathsf{M.CM}(\mathsf{Enc}(\mathbf{C})), \quad \mathbf{AB} = \mathbf{C}.$$

Thus a matrix commitment binds the verifier to the row-wise codeword matrix rather than to the raw matrix alone. Since $\mathcal{C}$ is systematic, the raw matrix is also determined by the first k coordinates of each committed row. Proving membership in $\mathcal{R}_{\mathsf{orig}}$ directly inside a SNARK would require $\mathcal{O}(k^3)$ multiplication constraints, which is exactly what we aim to avoid.

Reduction 1: Freivalds' randomization ($\mathcal{R}_{\mathsf{orig}} \rightsquigarrow \mathcal{R}_{\mathsf{Freivalds}}$). Using the verifier's random challenge $r \in \mathbb{F}$, define $\mathbf{r} = (1, r, r^2, \ldots, r^{k-1})$ and introduce three intermediate vectors

$$\mathbf{x} = \mathbf{rA}, \qquad \mathbf{y} = \mathbf{xB}, \qquad \mathbf{z} = \mathbf{rC}.$$

The Freivalds relation checks

$$\mathbf{rA} = \mathbf{x}, \qquad \mathbf{xB} = \mathbf{y}, \qquad \mathbf{rC} = \mathbf{z}, \qquad \mathbf{y} = \mathbf{z}.$$

By the Schwartz–Zippel lemma, if $\mathbf{AB} \neq \mathbf{C}$, then the final equality fails except with probability at most $(k-1)/|\mathbb{F}|$ over the choice of r. This removes the cubic matrix-product check, but checking the three vector-matrix products directly

would still require quadratic circuit work. No commitment to $\mathbf{x}, \mathbf{y}, \mathbf{z}$ is needed at this algebraic step; those commitments are introduced next only to make the sampled proximity checks non-adaptive.

Reduction 2: Proximity lifting ($\mathcal{R}_{\mathsf{Freivalds}} \rightsquigarrow \mathcal{R}_{\mathsf{Prox}}$). We now encode the matrices and intermediate vectors using a systematic $\mathbb{F}$-linear code $\mathcal{C} : \mathbb{F}^k \to \mathbb{F}^n$ with relative distance δ, and replace the global vector-matrix checks with *local* consistency checks at random positions. Let $I = (j_1, \ldots, j_t) \leftarrow\$\ [n]^t$ be the ordered tuple of queried column indices sampled independently with replacement, and let $\mathcal{M} = \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$ denote the matrix set and $\mathcal{W} = \{\mathbf{x}, \mathbf{y}, \mathbf{z}\}$ the intermediate-vector set. Let $\mathcal{S} = \mathcal{M} \cup \mathcal{W}$ be the set of objects opened at the sampled positions. The matrix roots $\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C$ are certified before r is sampled. After seeing r but before seeing I, the prover commits to $\mathbf{x}, \mathbf{y}, \mathbf{z}$ by sending message commitments μ_x, μ_y, μ_z and encoded-vector roots $\mathsf{cm}_x, \mathsf{cm}_y, \mathsf{cm}_z$. This timing is necessary: without a pre-query commitment, a malicious prover could choose intermediate vectors after seeing I and make them agree only at the sampled coordinates.

The *public instance* is

$$\mathsf{x} = \left(\{\mathsf{cm}_\chi\}_{\chi\in\mathcal{S}},\ \mu_x, \mu_y, \mu_z,\ r,\ I,\ \{\Gamma_{\chi,j}\}_{\chi\in\mathcal{S},\, j\in I}\right)$$

and the private data used for the circuit checks is

$$\mathsf{w} = (\mathbf{x},\ \mathbf{y},\ \mathbf{z},\ \{\omega_v\}_{v\in\mathcal{W}},\ \left\{a_{\chi,j}, \rho_{\chi,j}\right\}_{j\in I,\, \chi\in\mathcal{S}}).$$

The auxiliary opening/linking proof data is

$$\mathsf{aux}_I = \left(\left\{L_{\chi,j}, \pi_{\chi,j}\right\}_{j\in I,\, \chi\in\mathcal{S}}, \sigma_I\right).$$

For a matrix commitment, $a_{\chi,j} \in \mathbb{F}^k$ is the sampled encoded column $\mathsf{Enc}(\chi)^{(j)}$ used by the fold equation. For a vector commitment, $a_{\chi,j} \in \mathbb{F}$ is the sampled encoded coordinate. The resulting proximity-checking relation, denoted $\mathcal{R}_{\mathsf{Prox}}$, requires $(\mathsf{x}, \mathsf{w}, \mathsf{aux}_I)$ to satisfy all of the following clauses. In our implementation, clauses (MSG)–(EQ) are checked by Groth16; clauses (PATH) and (LINK) are checked by AuxV over the same transcript and public handles.

$$\forall v \in \mathcal{W}:\ \ \mathsf{Msg.Verify}(\mu_v, v, \omega_v) = 1, \qquad \text{(MSG)}$$
$$\forall \chi \in \mathcal{S},\ j \in I:\ \ \mathsf{Wire.Verify}(\mathsf{pp}, \Gamma_{\chi,j}, a_{\chi,j}, \rho_{\chi,j}) = 1, \qquad \text{(WIRE)}$$
$$\forall v \in \mathcal{W},\ j \in I:\ \ a_{v,j} = \mathsf{Enc}(v)_j, \qquad \text{(RS)}$$
$$\forall j \in I:\ \ a_{x,j} = \mathsf{Fold}(a_{A,j},\ \mathbf{r}), \qquad \text{(FOLD-A)}$$
$$\forall j \in I:\ \ a_{y,j} = \mathsf{Fold}(a_{B,j},\ \mathbf{x}), \qquad \text{(FOLD-B)}$$
$$\forall j \in I:\ \ a_{z,j} = \mathsf{Fold}(a_{C,j},\ \mathbf{r}), \qquad \text{(FOLD-C)}$$
$$\forall j \in I:\ \ a_{y,j} = a_{z,j}, \qquad \text{(EQ)}$$
$$\forall \chi \in \mathcal{S},\ j \in I:\ \ \mathsf{Open.Verify}(\pi_{\chi,j}, L_{\chi,j}, \mathsf{cm}_\chi, j) = 1 \qquad \text{(PATH)}$$
$$\mathsf{LinkV}(\mathsf{x}, I, \left((\Gamma_{\chi,j_q}, L_{\chi,j_q})\right)_{\chi\in\mathcal{S},\, 1\le q\le t}, \sigma_I) = 1 \qquad \text{(LINK)}$$

Here $\forall j \in I$ ranges over every sampled position in the ordered tuple, including repetitions. Equation (WIRE) is the critical bridge between the private Groth16 witness and the auxiliary opening world: the fold equations use $a_{\chi,j}$, and the circuit simultaneously proves that this same value opens the public handle $\Gamma_{\chi,j}$. Equation (PATH) uses the selected position-opening verifier on the compact leaf handle $L_{\chi,j}$. For the transparent Merkle instantiation, this is Merkle.Verify with the queried index included in the leaf domain. Equation (LINK) is the backend check that links the SNARK-side handle $\Gamma_{\chi,j}$ to the authenticated leaf handle $L_{\chi,j}$. Equation (MSG) is what ties the vectors used by the circuit to the message commitments that were fixed before I was sampled. Sending μ_x, μ_y, μ_z alone would not force the circuit witness to use the committed vectors. Equation (RS) computes the sampled RS/code evaluation for each intermediate vector inside the circuit. In the $\mathsf{FastMatMul}_{\mathsf{QA}}$ artifact, the message opening, wire-handle openings, sampled RS evaluations, and fold equations are included in the Groth16 relation, while Merkle path verification and sampled linking are auxiliary checks bound to the same transcript. The security proof uses only message-commitment binding, sampled witness-to-leaf link consistency, and position binding of the opening layer.

The critical property is that the Groth16 relation involves only $\mathcal{O}(|I| \cdot k)$ constraints for the fold operations, sampled RS evaluations, wire-handle openings, and message-opening checks. Merkle authentication and sampled-linking verification are auxiliary online checks whose cost is reported separately. Since $|I|$ is a security-parameter-sized constant independent of k, the in-circuit checking relation is *linear in* k.

Intermediate-binding cost. The message checks in (MSG) force the prover to commit to fixed vectors $\mathbf{x}, \mathbf{y}, \mathbf{z}$ before the query tuple is sampled. The wire and link checks in (WIRE) and (LINK) then force each sampled external opening to equal the value used by the in-circuit fold relation; for intermediate vectors, (RS) also forces this value to be the corresponding code evaluation $\mathsf{Enc}(v)_j$. Without the pre-query message commitment, a cheating prover could choose sampled openings first and then interpolate a vector that matches only those positions, defeating the distance-based soundness argument. Analogously, the input matrix commitments $\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C$ must be well-formed commitments to row-wise encodings of $\mathbf{A}, \mathbf{B}, \mathbf{C}$. In our model this is part of the public input-commitment interface, not a property re-proved by the multiplication protocol itself. For Reed–Solomon-style encodings, a sampled coordinate $\mathsf{Enc}(v)_j$ can be computed by evaluating the degree-$(k-1)$ message polynomial at the queried domain point, costing $\mathcal{O}(k)$ constraints per sampled coordinate. Thus the sampled intermediate checks contribute $\mathcal{O}(tk)$ constraints. A fallback backend that proves full dense codeword validity inside the circuit would instead require $\mathcal{O}(k(n-k))$ work for a dense generator matrix. We discuss the trade-off in Section 4.3.

4.2. Protocol Description

We now give the full specification of FastMatMul. Let $\mathcal{C} : \mathbb{F}^k \to \mathbb{F}^n$ be a systematic linear code with relative distance δ and M.CM denote the Merkle tree commitment. Let $\mathcal{M} = \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$ be the matrix set, $\mathcal{W} = \{\mathbf{x}, \mathbf{y}, \mathbf{z}\}$ the intermediate-vector set, and $\mathcal{S} = \mathcal{M} \cup \mathcal{W}$ the objects opened at sampled positions. The verifier is given certified input commitments $(\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C)$ to $\mathsf{Enc}(\mathbf{A}), \mathsf{Enc}(\mathbf{B}), \mathsf{Enc}(\mathbf{C})$. The protocol FastMatMul operates between a prover $\mathcal{P}$ and verifier $\mathcal{V}$ as shown in Figure 1.

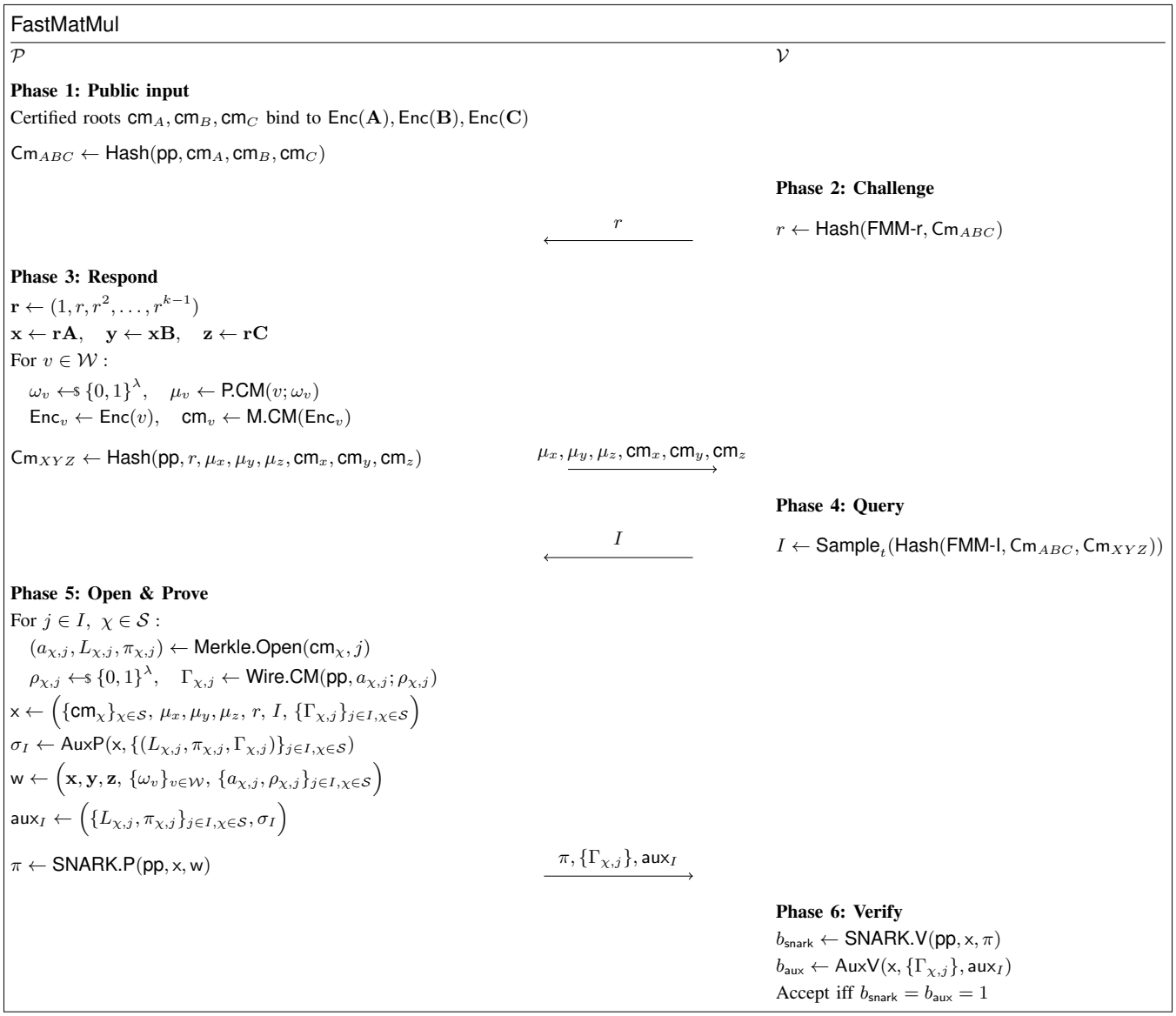

Figure 1. The FastMatMul protocol for verifiable matrix multiplication. $\mathcal{S} = \mathcal{M} \cup \mathcal{W}$ is the set of objects opened at sampled positions, $t = |I|$ is the query complexity, and x, w denote the public statement and witness for the inner SNARK.

Remark on the SNARK relation. The SNARK proof π attests to the in-circuit part of $\mathcal{R}_{\mathsf{Prox}}$ defined in Section 4.1; full transcript acceptance additionally requires the auxiliary opening and linking verifiers to accept. The implemented verifier checks three groups of conditions, split between the Groth16 circuit and auxiliary verifiers. First, constraint (MSG) ties the message commitments μ_x, μ_y, μ_z to the same $\mathbf{x}, \mathbf{y}, \mathbf{z}$ used in the fold checks. Constraint (WIRE) makes each sampled value used by the circuit open the public wire handle $\Gamma_{\chi,j}$, and constraints (RS)–(EQ) verify the sampled RS, fold, and equality checks at each $j \in I$ inside the Groth16 circuit. Second, the auxiliary backend checks (LINK), linking each public wire handle $\Gamma_{\chi,j}$ to the authenticated external leaf handle $L_{\chi,j}$. Third, the auxiliary opening verifier checks (PATH) for all Merkle opening proofs over the leaf handles. The public statement x consists of the commitment roots $\{\mathsf{cm}_\chi\}_{\chi\in\mathcal{S}}$, the intermediate message commitments μ_x, μ_y, μ_z, the challenge r, the query tuple I, and the sampled wire handles $\{\Gamma_{\chi,j}\}$. The witness w consists of the intermediate vectors, openings of their message commitments, the sampled matrix columns or vector leaves used by the fold equations, and the wire-handle opening randomness.

Non-interactive variant. The protocol as described is a three-round public-coin interactive protocol. By applying the Fiat–Shamir heuristic [5] in the random oracle model, it can be made non-interactive using domain-separated transcript hashes:

$$\mathsf{Cm}_{ABC} \leftarrow \mathsf{Hash}(\mathsf{pp}, \mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C),$$
$$r \leftarrow \mathsf{Hash}(\text{FMM-r}, \mathsf{Cm}_{ABC}).$$

and

$$\mathsf{Cm}_{XYZ} \leftarrow \mathsf{Hash}(\mathsf{pp}, r, \mu_x, \mu_y, \mu_z, \mathsf{cm}_x, \mathsf{cm}_y, \mathsf{cm}_z),$$
$$T_{\mathsf{qry}} \leftarrow (\mathsf{Cm}_{ABC}, \mathsf{Cm}_{XYZ}),$$
$$I \leftarrow \mathsf{Sample}_t(\mathsf{Hash}(\mathsf{FMM\text{-}I}, T_{\mathsf{qry}})) .$$

The resulting non-interactive proof consists of the commitment roots, the intermediate message commitments, the transcript-derived challenges, the sampled wire handles $\{\Gamma_{\chi,j}\}$, and the SNARK proof π. Merkle leaf handles, authentication paths, and sampled-linking material are auxiliary proof data verified by the corresponding backend. Raw sampled matrix columns are not serialized as proof data; they remain private Groth16 witness values bound to the public wire handles.

Security of the non-interactive variant. A potential concern with the Fiat–Shamir transform for proximity-based protocols is a *grinding attack*: a malicious prover could try many transcript prefixes until the derived challenge r or query tuple I is favorable. The random-oracle theorem accounts for this by the standard $\epsilon_{\mathsf{FS}}(q_H)$ loss in Theorem 5.1. For intuition, consider grinding only the intermediate commitments after r is fixed. Suppose the adversary has committed to $(\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C)$ (and hence to fixed encoded matrices) and, after receiving r, is searching for intermediate message commitments and encoded-vector roots such that the transcript-derived query tuple I avoids the disagreement set D of some fold or equality check. If any of the four checks fails globally, then $|D| \geq \delta n$ by the minimum-distance property. For a single random query $j \in [n]$, the probability of $j \notin D$ is at most $1 - \delta$; for t independent queries it is $(1 - \delta)^t$. Choosing t as in Section 5 makes the union-bounded proximity term $4(1-\delta)^t$ at most $2^{-\lambda}$. Each trial of the grinding attack produces an independently random I in the random oracle model because the query hash binds the full transcript, so the probability that q grinding attempts yield a favorable I is at most $q \cdot 4(1 - \delta)^t$. For any PPT adversary, q is polynomially bounded in λ, giving a total success probability of $\mathsf{negl}(\lambda)$. Hence the non-interactive protocol retains the soundness bound of Theorem 5.1 in the random oracle model, plus the stated Fiat–Shamir loss.

Zero-knowledge extension. The protocol as stated is a *verifiable computation* protocol and does not conceal the inputs $\mathbf{A}, \mathbf{B}, \mathbf{C}$ from the verifier. We do not claim zero knowledge for FastMatMul. A zero-knowledge variant would require replacing the Merkle tree commitments with hiding commitments, using a zero-knowledge SNARK backend for π, and masking the intermediate vectors and sampled openings so that the transcript does not reveal linear information about the committed matrices. The auxiliary opening/linking material would also need a separate simulation argument. We therefore treat zero knowledge as future work rather than as a property of the construction analyzed in this paper.

4.3. Complexity Analysis

We analyze the costs of FastMatMul.

Prover computation. We separate the cost of producing the claimed output matrix and certified input roots from the cost of proving that the certified roots satisfy the claim. Computing $\mathbf{C} = \mathbf{AB}$ is application work shared by all approaches and is not part of the checking-layer comparison. Likewise, producing certified roots for raw input matrices is an upstream cost: with linear-time codes it takes $\mathcal{O}(kn)$ field work for a $k \times k$ matrix, plus handle and hash work to form the root. After those roots are fixed, the prover's dominant online checking-layer costs are:

(i) *Vector computation*: Computing $\mathbf{x}, \mathbf{y}, \mathbf{z}$ costs $\mathcal{O}(k^2)$ in total.
(ii) *Intermediate-vector roots*: Encoding $\mathbf{x}, \mathbf{y}, \mathbf{z}$ and forming their roots costs $\mathcal{O}(n)$ for constant-rate linear codes.
(iii) *SNARK proving*: The circuit size is $\mathcal{O}(t \cdot k + E_{\mathsf{msg}}(k))$ for the fold checks, sampled RS evaluations, wire-handle openings, and message-opening constraints. Merkle and sampled-linking work is auxiliary and is included in the reported proving, verification, and online-byte breakdowns.

Verifier computation. The verifier performs SNARK verification together with any commitment/linking checks required by the chosen backend. For a pairing-based SNARK whose public input consists only of the roots, challenge, and query indices, the core SNARK verification is $\mathcal{O}(1)$. For transparent or linking-heavy backends, verification may scale with the public input or linking proof bytes; the concrete behavior is reported in Section 6.

Communication. The communication depends on the commitment layer. With the sampled-opening backend evaluated in this paper, the proof contains the SNARK proof, message commitments, public wire handles, authentication paths, and auxiliary linking material for the queried objects. The raw sampled matrix columns contain $\mathcal{O}(tk)$ field elements, but they are Groth16 witness values rather than serialized proof bytes. The serialized auxiliary material is compact: wire/leaf handles, Merkle paths, and link proofs scale as $\mathcal{O}(t \log n + E_{\mathsf{link}}(k))$ for fixed security parameter. This accounting explains the sub-megabyte online proof sizes reported in Section 6 and Appendix C; making the sampled columns public would instead add $\Theta(tk)$ field elements.

Circuit size comparison. Table 3 summarizes the asymptotic costs. Naïve arithmetization uses $\mathcal{O}(k^3)$ constraints. Freivalds' algorithm alone uses $\mathcal{O}(k^2)$. The online checker achieves

$$\mathcal{O}\big(tk + E_{\mathsf{msg}}(k)\big) \text{ in-circuit work}$$

plus $\mathcal{O}(t \log n + E_{\mathsf{link}}(k))$ auxiliary opening/linking work. The in-circuit term covers the fold checks, sampled RS evaluations, and message-opening checks. The auxiliary term covers Merkle authentication and sampled-linking verification over compact sampled handles. The older notation $E_{\mathsf{cc}}(k)$ denotes the combined message-binding/linking cost, with $E_{\mathsf{cc}}(k) = E_{\mathsf{msg}}(k) + E_{\mathsf{link}}(k)$ for the current artifact. For dense general codes, a full in-circuit codeword-validity fallback gives $E_{\mathsf{cc}}(k) = \mathcal{O}(k(n-k))$, which is $\mathcal{O}(k^2)$ at

Component	Prover	Verifier
Certified input roots	upstream $\mathcal{O}(kn)$	certified public input
Intermediate vectors	$\mathcal{O}(k^2)$	—
Vector roots	$\mathcal{O}(n)$	—
SNARK circuit size	$\mathcal{O}(tk + E_{\mathsf{msg}}(k))$	—
Auxiliary opening/linking	$\mathcal{O}(t \log n + E_{\mathsf{link}}(k))$	backend-dependent
Core SNARK verification	—	$\mathcal{O}(1)$
Online checking total	$\mathcal{O}(k^2+n+tk+E_{\mathsf{msg}}+E_{\mathsf{link}})$	backend-dependent

TABLE 3. ASYMPTOTIC COSTS AFTER SEPARATING UPSTREAM INPUT-ROOT CERTIFICATION FROM ONLINE MULTIPLICATION CHECKING. THE ONLINE TOTAL EXCLUDES NATIVE COMPUTATION OF $\mathbf{C} = \mathbf{AB}$ AND RAW INPUT CERTIFICATION. k: MATRIX DIMENSION; n: CODEWORD LENGTH; t: QUERY COUNT; E_{msg} AND E_{link} ARE BACKEND COSTS.

constant rate. For the sampled-opening backend evaluated in this paper, the online sampled encoding work is already included in the $\mathcal{O}(tk)$ term. Setting

$$t = \left\lceil \frac{\lambda + 2}{\log_2(1/(1-\delta))} \right\rceil$$

then gives $\mathcal{O}(k)$ in-circuit work and $\mathcal{O}(\log k)$ serialized auxiliary opening material for fixed security parameter, with concrete sizes discussed in Section 6.

5. Security Analysis

We prove that the protocol FastMatMul (Figure 1) constitutes a secure interactive argument for the matrix multiplication relation $\mathcal{R}_{\mathsf{orig}}$ relative to certified encoded input commitments and the sampled-opening backend of Definition 4.1. Soundness and completeness are established in Theorem 5.1 below; knowledge soundness is discussed separately in Remark 2. As discussed in Section 4.1, the public input commitments $\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C$ are assumed to bind to row-wise encodings of fixed matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$. This is the committed-input model used by code-based proof systems such as Brakedown [6] and Orion [7]; FastMatMul verifies the multiplication relation among these committed objects. For the intermediate vectors, the prover commits to the message vectors $\mathbf{x}, \mathbf{y}, \mathbf{z}$ before the query tuple is sampled. At each sampled coordinate, the SNARK binds the matrix column or vector leaf it uses to a public wire handle $\Gamma_{\chi,j}$. The auxiliary verifier authenticates a compact external Merkle leaf handle $L_{\chi,j}$ and checks that it is linked to the SNARK-side handle. For intermediate vectors, the SNARK also computes $\mathsf{Enc}(v)_j$ internally before opening the wire handle.

Scope of the security claim. The theorem is a statement-soundness result for committed matrix multiplication, not a standalone argument-of-knowledge or zero-knowledge theorem. The public objects are the certified encoded input commitments and the transcript-derived challenges; the committed objects are the row-wise encodings of $\mathbf{A}, \mathbf{B}, \mathbf{C}$ and the intermediate encoded vectors. The backend supplies message binding before I and sampled consistency between the Groth16 wire handles and the external Merkle openings. Witness extraction and hiding require the additional composition assumptions described in Remark 2 and are outside the main theorem. In short, $\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C$ are certified public roots fixed before r; μ_x, μ_y, μ_z and $\mathsf{cm}_x, \mathsf{cm}_y, \mathsf{cm}_z$ are fixed before I; sampled raw columns are private Groth16 witness values linked to authenticated leaf handles.

Soundness-critical ordering. The proof uses the following timing invariants. First, the certified input roots $\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C$ are fixed before the Freivalds challenge r is sampled. Second, the intermediate message commitments and encoded-vector roots are fixed before the query tuple $I = (j_1, \ldots, j_t) \leftarrow\$ [n]^t$ is sampled; conditioned on an accepting message-opening proof, each vector $\mathbf{x}, \mathbf{y}, \mathbf{z}$ is fixed independently of the sampled positions, and accepted auxiliary linking proofs bind sampled external openings to the corresponding Groth16 wire handles. For intermediate vectors, the message openings and in-circuit RS checks bind those handles to the corresponding coordinates of $\mathsf{Enc}(\mathbf{x}), \mathsf{Enc}(\mathbf{y}), \mathsf{Enc}(\mathbf{z})$. Third, the entries of I are sampled independently with replacement, as defined in Section 3.1. If an implementation samples without replacement, the proximity failure probability only decreases; the theorem states the simpler with-replacement bound.

Backend failure event. The backend bad event is the only non-standard composition point. After the transcript prefix fixes $(\mathsf{Cm}_{ABC}, r, \mathsf{Cm}_{XYZ}, I)$, the adversary must output Groth16 handles, Merkle openings, and an auxiliary linking proof for the sampled objects. A backend failure occurs if all these checks accept while some value used in the Groth16 fold relation is not the value authenticated at the corresponding leaf position. Definition 4.1 charges this event to ϵ_{cc}; Appendix A expands the event for the concrete backend. For the artifact backend summarized in Section 4, the sampled-linking component contributes $\epsilon_{\mathsf{link}} \leq \epsilon_{\mathsf{ped}} + \epsilon_{\mathsf{QA}} + Q/|\mathbb{F}|$, where $Q = |\mathcal{S}|t$ is the number of sampled handle pairs. Here ϵ_{ped} is the binding error of the algebraic handle parameters, ϵ_{QA} is the soundness error of the batched QA-NIZK relation, and the $Q/|\mathbb{F}|$ term is the standard randomized-batch failure probability after the batch statement is fixed.

Theorem 5.1 (Security)**.** *The protocol FastMatMul satisfies the following properties:*

(i) ***Perfect completeness.*** *If $\mathbf{AB} = \mathbf{C}$, an honest prover always makes the verifier accept.*

(ii) ***Interactive computational soundness.*** *For any PPT adversary $\mathcal{P}^*$ given certified public input commitments to $\mathbf{A}, \mathbf{B}, \mathbf{C}$, let Acc be the event that the verifier accepts a false committed matrix statement. Then:*

$$\Pr[\mathsf{Acc} \mid \mathbf{AB} \neq \mathbf{C}] \leq \epsilon_{\mathsf{SNARK}}(\lambda) + \epsilon_{\mathsf{cc}}(\lambda) + \epsilon_{\mathsf{bind}}(\lambda) + \frac{k-1}{|\mathbb{F}|} + 4(1-\delta)^t,$$

Here $t = |I|$ is the query complexity and δ is the relative minimum distance of $\mathcal{C}$. The terms $\epsilon_{\mathsf{SNARK}}$, ϵ_{cc}, and ϵ_{bind} denote, respectively, the soundness error of the SNARK backend, the message-binding/linking consistency error of the selected intermediate backend, and the union-bounded position-binding error of the opening layer over all roots and openings used in the protocol. In the notation of Definition 4.1, $\epsilon_{\mathsf{cc}} \le \epsilon_{\mathsf{msg}} + \epsilon_{\mathsf{link}}$. For the artifact backend, $\epsilon_{\mathsf{link}} \le \epsilon_{\mathsf{ped}} + \epsilon_{\mathsf{QA}} + |\mathcal{S}|t/|\mathbb{F}|$; Appendix B gives the full batching equation.

(iii) ***Fiat–Shamir soundness.*** *In the random oracle model, the non-interactive variant described in Section 4.2 satisfies the same bound with an additional Fiat–Shamir loss term $\epsilon_{\mathsf{FS}}(q_H)$, where q_H is the adversary's number of random-oracle queries.*

Corollary 5.2 (Evaluated backend)**.** *For the backend used in the artifact, Theorem 5.1 applies with*

$$\epsilon_{\mathsf{cc}} \le \epsilon_{\mathsf{msg}} + \epsilon_{\mathsf{ped}} + \epsilon_{\mathsf{QA}} + |\mathcal{S}|t/|\mathbb{F}|.$$

The bound assumes the message commitment is binding, the algebraic handle parameters are binding, the QA-NIZK is sound for the stated linear batch relation, and the batch randomizer is sampled after the handle-pair statement is fixed. In the non-interactive artifact, the Fiat–Shamir loss term $\epsilon_{\mathsf{FS}}(q_H)$ covers both the protocol challenges (r, I) and the backend batch randomizer, using separate domain tags.

The reported scaling rows use $t = 128$; if $\delta = 1/2$, this gives proximity term 2^{-126}, while a conservative per-term 2^{-131} target uses $t = 133$. A strict deployment must also instantiate the SNARK, message commitment, sampled-linking backend, and hash parameters so their union-bounded errors fit the aggregate security budget.

Proof sketch. Completeness follows from linearity of the Freivalds reduction and of the encoding: honest openings satisfy every sampled fold and equality check. For soundness, condition away SNARK, message/linking, and position-binding failures. Then the input roots fix $\mathbf{A}, \mathbf{B}, \mathbf{C}$ before r, the intermediate commitments fix $\mathbf{x}, \mathbf{y}, \mathbf{z}$ before I, and accepted sampled openings are the same values used by the Groth16 fold relation. Schwartz–Zippel gives the Freivalds term $(k-1)/|\mathbb{F}|$; conditioned on Freivalds not failing, at least one of the four encoded relations is false, so t random coordinates miss its disagreement set with probability at most $(1-\delta)^t$. Union-bounding over the four relations gives the proximity term. Appendix A gives the detailed proof and the Fiat–Shamir random-oracle reduction.

Remark 1 (Fiat–Shamir soundness). The proof of the main bound in Theorem 5.1 is for the public-coin interactive protocol. The non-interactive variant in Section 4.2 derives both r and I from domain-separated hashes of the full transcript prefix. The artifact backend also derives its batch randomizer with a separate domain tag after the handle-pair batch statement is fixed. In the random oracle model, the same proof applies with the standard Fiat–Shamir loss $\epsilon_{\mathsf{FS}}(q_H)$, where q_H is the adversary's number of random-oracle queries. This term accounts for grinding over transcript prefixes and is negligible for polynomial q_H when the challenge and query hashes have λ-bit security.

Remark 2 (Knowledge Soundness). The theorem above is a statement-soundness result relative to certified public input commitments. A full *argument of knowledge* can be obtained by composing FastMatMul with an extractable input-commitment interface and a knowledge-sound SNARK. In that setting, the input-commitment extractor yields openings of $\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C$ to row-wise encoded matrices, while the SNARK extractor yields the intermediate vectors and queried openings used in $\mathcal{R}_{\mathsf{Prox}}$. The same soundness argument then implies that any prover accepted with non-negligible probability is bound, except with the stated error, to matrices satisfying $\mathbf{AB} = \mathbf{C}$. We treat this as an orthogonal composition issue and focus the main theorem on the multiplication-checking protocol.

6. Implementation and Evaluation

We implement the multiplication-checking layer of FastMatMul and evaluate the current artifact, denoted $\mathsf{FastMatMul}_{\mathsf{QA}}$, against an optimized SNARK-in-circuit Freivalds baseline [1]. $\mathsf{FastMatMul}_{\mathsf{QA}}$ uses multi-column Merkle openings, a Groth16 circuit proof, and the sampled-opening backend from Section 4. All main experiments use a 256-bit prime field, rate $\rho = 1/2$, codeword length $n = 2k$, and $t = 128$ sampled positions. The experiments were run on a Linux server with an AMD EPYC 7B13 CPU (32 cores) and 251.69 GiB RAM.

These measurements include the online sampled-handle path described in Section 4: message commitments fix $\mathbf{x}, \mathbf{y}, \mathbf{z}$ before I, the circuit binds every sampled matrix column or vector leaf to a public wire handle, Merkle openings authenticate compact external leaf handles, and auxiliary linking checks connect the two views. The raw sampled matrix columns are SNARK witness data, not serialized auxiliary proof bytes. All byte counts in this section are online proof bytes: they exclude raw input certification for $\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C$, proving/verifying keys, serialized CRS data, and generator material. They should not be read as complete end-to-end deployment costs from raw matrices. Appendix C records parameter and measurement coverage details.

Primary matrix benchmark. Figure 2 shows the main scaling behavior. Freivalds constraints grow by approximately $4\times$ with each doubling of k, matching the expected $\mathcal{O}(k^2)$ scaling. In contrast, $\mathsf{FastMatMul}_{\mathsf{QA}}$ grows by approximately $2\times$, matching the $\mathcal{O}(k)$ checking-layer scaling of the sampled-opening backend. The crossover in absolute constraint count occurs at $k = 256$. At $k = 1{,}024$, $\mathsf{FastMatMul}_{\mathsf{QA}}$ uses $419{,}139$ constraints versus $3{,}156{,}298$ for Freivalds, a $7.53\times$ reduction. At $k = 4{,}096$, $\mathsf{FastMatMul}_{\mathsf{QA}}$ uses $1{,}667{,}305$ constraints versus $50{,}495{,}818$ for Freivalds, a $30.29\times$ reduction.

The same benchmark shows a proving-time crossover at $k = 256$. At $k = 4{,}096$, $\mathsf{FastMatMul}_{\mathsf{QA}}$ proves in $55.92\,\mathrm{s}$ versus $405.04\,\mathrm{s}$ for Freivalds, a $7.24\times$ speedup.

$\log k$	Constraints		Prove Time (s)		Verify Time		FastMatMul$_{QA}$ Online Proof
	FastMatMul$_{QA}$	Freivalds	FastMatMul$_{QA}$	Freivalds	FastMatMul$_{QA}$	Freivalds	
7	55,059	49,610	0.78	0.58	0.01 s	1 ms	56,388 B
8	106,905	197,194	1.57	2.03	0.01 s	1 ms	69,028 B
9	210,985	788,810	3.37	6.89	0.01 s	1 ms	89,188 B
10	419,139	3,156,298	7.90	25.63	0.01 s	2 ms	108,548 B
11	835,065	12,622,154	20.91	102.07	0.01 s	2 ms	126,788 B
12	1,667,305	50,495,818	55.92	405.04	0.01 s	2 ms	146,788 B
13	3,331,779	—	181.29	—	0.01 s	—	167,748 B

TABLE 4. PRIMARY MATRIX-MULTIPLICATION BENCHMARK FOR FASTMATMUL$_{QA}$ AND THE OPTIMIZED FREIVALDS SNARK BASELINE. FASTMATMUL$_{QA}$ IS THE CURRENT IMPLEMENTATION OF THE SAMPLED-OPENING BACKEND WITH MULTI-COLUMN MERKLE OPENINGS. PROVING TIME EXCLUDES NATIVE MATRIX MULTIPLICATION AND SETUP; THESE ARE RECORDED SEPARATELY IN THE ARTIFACT. THE TABLE MEASURES THE ONLINE MULTIPLICATION-CHECKING LAYER, INCLUDING WIRE-HANDLE OPENINGS, SAMPLED RS EVALUATIONS, MERKLE OPENINGS, AND AUXILIARY OPENING/LINKING CHECKS; ONLINE PROOF BYTES EXCLUDE RAW SAMPLED MATRIX-COLUMN WITNESS VALUES, RAW INPUT CERTIFICATION FOR CM_A, CM_B, CM_C, AND SETUP/KEY/CRS MATERIAL.

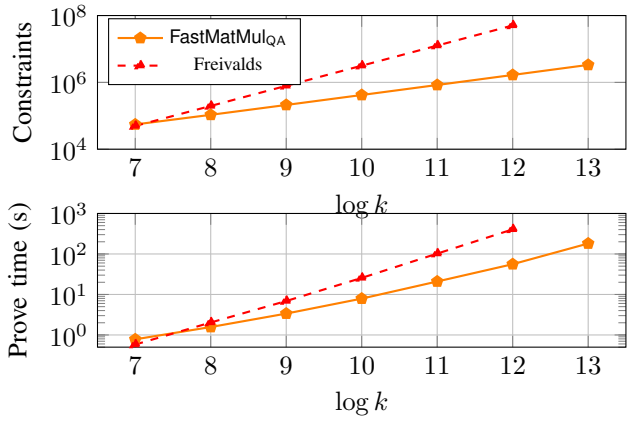

Figure 2. Primary matrix benchmark. With the sampled-opening backend, FastMatMul$_{QA}$ follows the expected $\mathcal{O}(k)$ checking-layer growth, whereas the Freivalds SNARK baseline follows $\mathcal{O}(k^2)$ growth.

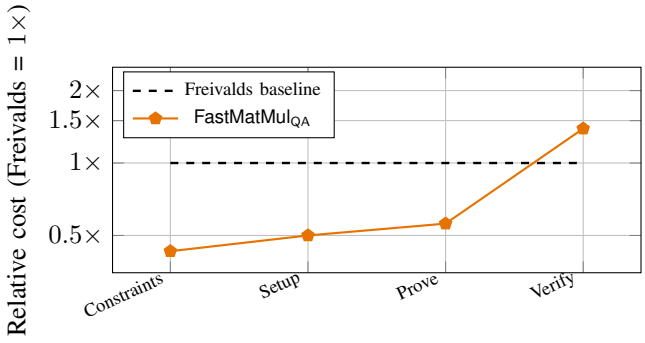

Figure 3. GPT-2 checking-layer case study at sequence length 1,024 with 36 matrix-multiplication claims over certified roots. Costs are normalized to the Freivalds baseline.

FastMatMul$_{QA}$ also completes $k = 8{,}192$ with 3,331,779 constraints and a 181.29 s proving time; Freivalds was not measured at that size in the current artifact.

Verification, online bytes, and GPT-2 workload. FastMatMul$_{QA}$ verification is nearly constant at about 0.01 s in the matrix benchmark, dominated by the auxiliary linking backend; Freivalds verification is faster because it checks only a direct Groth16 proof. The artifact therefore trades slower verification for smaller prover circuits and faster proving at medium and large dimensions. The FastMatMul$_{QA}$ online proof grows from 56,388 bytes at $k = 128$ to 146,788 bytes at $k = 4{,}096$ and 167,748 bytes at $k = 8{,}192$. At $k = 4{,}096, t = 128$, publishing the three sampled input-matrix columns alone would require about $3tk \cdot 32 \approx 50$ MB; Appendix C gives detailed byte, setup, and constraint accounting.

Metric	FastMatMul$_{QA}$	Freivalds	Ratio
Constraints	33.4M	76.8M	0.43×
Setup time	2,078.38 s	4,173.32 s	0.50×
Prove time	230.14 s	408.69 s	0.56×
Verify time	0.68 s	0.49 s	1.39×
FastMatMul$_{QA}$ online proof	3,355,428 B	—	—

TABLE 5. EXACT GPT-2 CHECKING-LAYER METRICS FROM THE CSV ARTIFACT FOR SEQUENCE LENGTH 1,024, 36 MATRIX-MULTIPLICATION CLAIMS, AND 5 COMMITMENT GROUPS.

Figure 3 and Table 5 report the GPT-2-style workload over certified roots. FastMatMul$_{QA}$ reduces constraints by 2.30× and proving time by 1.78×, with lower setup time in this experiment. Verification is 1.39× slower, so latency-sensitive deployments still need batching, amortization, or a more succinct opening/linking backend. The 3.36 MB GPT-2 online proof is larger than the single-matrix rows because it aggregates 36 multiplication claims and their sampled handles, Merkle authentication paths, and linking material.

Strict parameters and deployment scope. The measured rows use $t = 128$ to study scaling. If $\delta = 1/2$, a conservative per-term 2^{-131} proximity budget uses $t = 133$, increasing sample-dependent work by only 1.0391×. At $k = 4{,}096$, this extrapolates to at most 1.74 million constraints, 58.10 s proving time, and 153 KB online proof bytes. The current artifact does not include measured $t = 133$ rows; these strict numbers are extrapolations from the measured $t = 128$ rows. Online-byte figures exclude raw input certification, proving/verifying keys, and serialized CRS or generator material. End-to-end deployments preserve the measured prover-time win only if amortized root certification costs stay below the

observed margins: $349.12\,\mathrm{s}$ for $k = 4{,}096$ and $178.55\,\mathrm{s}$ for the GPT-2 row. The CSV artifact reports point estimates and aggregate setup/proving/verification times; repeated-run variance, confidence intervals, peak memory, and serialized key/CRS/generator sizes remain unmeasured.

7. Conclusion and Related Work

Our work complements verifiable ML systems such as SafetyNets [4], Mystique [2], zkCNN [18], vCNN [19], and zkML [3]; sumcheck/GKR-based VC [20]; dedicated matrix-verification protocols [21]–[24]; and code-based SNARK/proximity systems [6], [7], [25], [26]. Unlike full-circuit ML systems or polynomial-commitment matrix arguments [27], FastMatMul is a dedicated multiplication checking layer over certified encoded roots and is orthogonal to succinct backends, including universal/updatable-SRS and inner-product-style systems [12], [13], [28]. The security theorem accounts for SNARK soundness, message/leaf binding, sampled-linking consistency, Freivalds failure, and proximity-sampling error.

The artifact confirms the expected separation. At $k = 4{,}096$, $\mathsf{FastMatMul}_{\mathsf{QA}}$ gives a $30.29\times$ constraint reduction and a $7.24\times$ proving-time speedup over Freivalds-in-SNARK; on the GPT-2 case study it gives a $2.30\times$ constraint reduction and a $1.78\times$ proving-time speedup over certified roots. The remaining limitation is explicit: the artifact measures the online checking layer but assumes certified encoded input roots for raw matrices. Closing that pipeline, reporting key/CRS sizes, reducing verification overhead, and batching many multiplications are the main deployment steps.

Ethics considerations

This work uses local benchmark data and involves no human subjects, live systems, or vulnerability disclosure.

LLM usage considerations

LLMs were used only for editorial assistance; the authors checked all technical claims, proofs, experiments, and citations.

References

[1] R. Freivalds, "Probabilistic machines can use less running time," in *Information Processing 77 (Proceedings of IFIP Congress 77)*. Amsterdam, The Netherlands: North-Holland, 1977, pp. 839–842.

[2] C. Weng, K. Yang, X. Xie, J. Katz, and X. Wang, "Mystique: Efficient conversions for {Zero-Knowledge} proofs with applications to machine learning," in *30th USENIX Security Symposium (USENIX Security 21)*. Berkeley, CA, USA: USENIX Association, 2021, pp. 501–518.

[3] B.-J. Chen, S. Waiwitlikhit, I. Stoica, and D. Kang, "Zkml: An optimizing system for ml inference in zero-knowledge proofs," in *Proceedings of the Nineteenth European Conference on Computer Systems*. New York, NY, USA: ACM, 2024, pp. 560–574.

[4] Z. Ghodsi, T. Gu, and S. Garg, "Safetynets: Verifiable execution of deep neural networks on an untrusted cloud," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[5] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Advances in Cryptology – CRYPTO '86*. Berlin, Heidelberg: Springer, 1987, pp. 186–194.

[6] A. Golovnev, J. Lee, S. Setty, J. Thaler, and R. S. Wahby, "Brakedown: Linear-time and field-agnostic snarks for r1cs," in *Advances in Cryptology – CRYPTO 2023*. Cham: Springer, 2023, pp. 193–226.

[7] T. Xie, Y. Zhang, and D. Song, "Orion: Zero knowledge proof with linear prover time," in *Advances in Cryptology – CRYPTO 2022*. Cham: Springer, 2022, pp. 299–328.

[8] R. C. Merkle, "A certified digital signature," in *Advances in Cryptology – CRYPTO '89*. Berlin, Heidelberg: Springer, 1989, pp. 218–238.

[9] B. Parno, J. Howell, C. Gentry, and M. Raykova, "Pinocchio: Nearly practical verifiable computation," in *2013 IEEE Symposium on Security and Privacy*. New York, NY, USA: IEEE, 2013, pp. 238–252.

[10] J. Groth, "On the size of pairing-based non-interactive arguments," in *Advances in Cryptology – EUROCRYPT 2016*. Berlin, Heidelberg: Springer, 2016, pp. 305–326.

[11] R. S. Wahby, I. Tzialla, A. Shelat, J. Thaler, and M. Walfish, "Doubly-efficient zksnarks without trusted setup," in *2018 IEEE Symposium on Security and Privacy*. New York, NY, USA: IEEE, 2018, pp. 926–943.

[12] M. Maller, S. Bowe, M. Kohlweiss, and S. Meiklejohn, "Sonic: Zero-knowledge snarks from linear-size universal and updatable structured reference strings," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2019, pp. 2111–2128.

[13] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. Ward, "Marlin: Preprocessing zksnarks with universal and updatable srs," in *Advances in Cryptology – EUROCRYPT 2020*. Cham: Springer, 2020, pp. 738–768.

[14] S. Setty, "Spartan: Efficient and general-purpose zksnarks without trusted setup," in *Advances in Cryptology – CRYPTO 2020*. Cham: Springer, 2020, pp. 704–737.

[15] J. T. Schwartz, "Fast probabilistic algorithms for verification of polynomial identities," *Journal of the ACM*, vol. 27, no. 4, pp. 701–717, 1980.

[16] R. Zippel, "Probabilistic algorithms for sparse polynomials," in *Symbolic and Algebraic Computation (EUROSAM '79)*. Berlin, Heidelberg: Springer, 1979, pp. 216–226.

[17] C. S. Jutla and A. Roy, "Shorter quasi-adaptive NIZK proofs for linear subspaces," Cryptology ePrint Archive, Paper 2013/109, 2013. [Online]. Available: https://eprint.iacr.org/2013/109

[18] T. Liu, X. Xie, and Y. Zhang, "Zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2021, pp. 2968–2985.

[19] S. Lee, H. Ko, J. Kim, and H. Oh, "vcnn: Verifiable convolutional neural network based on zk-snarks," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 4254–4270, 2024.

[20] S. Goldwasser, Y. T. Kalai, and G. N. Rothblum, "Delegating computation: Interactive proofs for muggles," *Journal of the ACM*, vol. 62, no. 4, pp. 1–64, 2015.

[21] M. Cong, T. H. Yuen, and S.-M. Yiu, "zkmatrix: Batched short proof for committed matrix multiplication," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2024, pp. 289–305.

[22] M. Cong, T. H. Yuen, and S. M. Yiu, "Dualmatrix: conquering zksnark for large matrix multiplication," *Cybersecurity*, vol. 9, no. 1, p. 126, 2026.

[23] Y. Zhang, M. Zheng, X. Chen, J. Hu, W. Shi, L. Ju, Y. Solihin, and Q. Lou, "zkvc: Fast zero-knowledge proof for private and verifiable computing," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, IEEE. New York, NY, USA: IEEE, 2025, pp. 1–7.

[24] H. Bennett, K. Gajulapalli, A. Golovnev, and E. Warton, "Matrix multiplication verification using coding theory," *arXiv preprint arXiv:2309.16176*, 2023.

[25] S. Ames, C. Hazay, Y. Ishai, and M. Venkitasubramaniam, "Ligero: Lightweight sublinear arguments without a trusted setup," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2017, pp. 2087–2104.

[26] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Fast reed-solomon interactive oracle proofs of proximity," in *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*. Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018, pp. 14:1–14:17.

[27] A. Kate, G. M. Zaverucha, and I. Goldberg, "Constant-size commitments to polynomials and their applications," in *Advances in Cryptology – ASIACRYPT 2010*. Berlin, Heidelberg: Springer, 2010, pp. 177–194.

[28] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," in *2018 IEEE Symposium on Security and Privacy*. New York, NY, USA: IEEE, 2018, pp. 315–334.

[29] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Advances in Cryptology – CRYPTO '91*. Berlin, Heidelberg: Springer, 1991, pp. 129–140.

Appendix A. Detailed Proof of Theorem 5.1

Proof. We establish completeness and soundness for the interactive protocol, and then explain the Fiat–Shamir variant.

Perfect completeness. Suppose the prover is honest and $\mathbf{AB} = \mathbf{C}$. Let $\mathbf{D} \coloneqq \mathbf{AB} - \mathbf{C} = \mathbf{0}$. For any challenge vector $\mathbf{r}$, we have $\mathbf{rD} = \mathbf{0}$, so

$$\mathbf{y} = \mathbf{xB} = (\mathbf{rA})\mathbf{B} = \mathbf{r}(\mathbf{AB}) = \mathbf{rC} = \mathbf{z}.$$

For each intermediate vector $v \in \{\mathbf{x}, \mathbf{y}, \mathbf{z}\}$, the honest prover commits to the message vector before I is sampled, commits to the encoded leaf handles, opens public wire handles inside the Groth16 relation, and supplies accepted Merkle openings and CP-link proofs for the sampled coordinates. For any queried index $j \in [n]$, linearity of $\mathcal{C}$ gives

$$\mathsf{Fold}(\mathsf{Enc}(\mathbf{A})^{(j)}, \mathbf{r}) = \mathsf{Enc}(\mathbf{rA})_j = \mathsf{Enc}(\mathbf{x})_j,$$

and analogously for the $\mathbf{B}$ and $\mathbf{C}$ folds. Since $\mathbf{y} = \mathbf{z}$, the equality check between their encoded openings also holds at every coordinate. All authenticated openings and backend checks are honest, so the verifier accepts with probability one.

Bad-event decomposition. Let Acc be the event that the verifier accepts a false committed matrix statement, and assume $\mathbf{AB} \neq \mathbf{C}$. Define $\mathsf{Bad}_{\mathsf{SNARK}}$ as acceptance of a false in-circuit SNARK instance, $\mathsf{Bad}_{\mathsf{cc}}$ as a message-binding or witness-to-leaf linking failure for an accepted sampled object, and $\mathsf{Bad}_{\mathsf{bind}}$ as two inconsistent accepted leaf-handle openings for the same root and position. For the sampled-opening part of $\mathsf{Bad}_{\mathsf{cc}}$, condition on the Groth16 extractor for the in-circuit relation: for every accepted wire check it obtains $(a_{\chi,j}, \rho_{\chi,j})$ such that $\mathsf{Wire.Verify}(\mathsf{pp}, \Gamma_{\chi,j}, a_{\chi,j}, \rho_{\chi,j}) = 1$. The CP-link batch tag is $T_I = \mathsf{Hash}(\mathsf{FMM\text{-}link\text{-}batch}, \mathsf{Cm}_{ABC}, \mathsf{Cm}_{XYZ}, I, P_I)$, where $P_I = \{(\chi, j, \Gamma_{\chi,j}, L_{\chi,j})\}_{\chi,j}$. Thus the auxiliary proof is tied to the same handle pairs that appear in the SNARK public statement and the Merkle openings. A sampled linking failure occurs if $\mathsf{Open.Verify}(\pi_{\chi,j}, L_{\chi,j}, \mathsf{cm}_\chi, j) = 1$ and $\mathsf{CPLink.Verify}(\mathsf{pp}, T_I, \{(\Gamma_{\chi,j}, L_{\chi,j})\}_{\chi,j}, \tau_I) = 1$, but for some sampled pair there is no leaf-opening randomness $\eta_{\chi,j}$ such that $L_{\chi,j}$ opens to the same $a_{\chi,j}$ extracted from the Groth16 witness. For the Pedersen handle instantiation this means there is no $\eta_{\chi,j}$ with

$$L_{\chi,j} = \eta_{\chi,j} H + \sum_{\ell=1}^{d_\chi} (a_{\chi,j})_\ell G_\ell,$$
$$\Gamma_{\chi,j} - L_{\chi,j} = (\rho_{\chi,j} - \eta_{\chi,j}) H.$$

Merkle position binding is charged separately to $\mathsf{Bad}_{\mathsf{bind}}$; outside that event, the accepted $L_{\chi,j}$ is the unique handle at position j under the pre-query root cm_χ. By the assumed backend properties,

$$\Pr[\mathsf{Bad}_{\mathsf{SNARK}}] \le \epsilon_{\mathsf{SNARK}}(\lambda),$$
$$\Pr[\mathsf{Bad}_{\mathsf{cc}}] \le \epsilon_{\mathsf{cc}}(\lambda),$$
$$\Pr[\mathsf{Bad}_{\mathsf{bind}}] \le \epsilon_{\mathsf{bind}}(\lambda).$$

Conditioned on the complement of these events, every accepting transcript has a valid SNARK witness, every accepted intermediate message commitment binds a single vector v before I is sampled, every sampled value used by the Groth16 fold relation opens a public wire handle, every accepted auxiliary opening is linked to that handle, and every accepted leaf handle is uniquely determined by its root and queried coordinate. The certified input roots are fixed before the Freivalds challenge is sampled, and the intermediate commitments are fixed before the sampled query tuple is sampled.

Freivalds randomization. Under the same conditioning, the matrices bound to $\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C$ are fixed and satisfy $\mathbf{AB} \neq \mathbf{C}$. Let $\mathbf{D} = \mathbf{AB} - \mathbf{C} \neq 0$. There exists a column $\mathbf{d}_j$ of $\mathbf{D}$ with $\mathbf{d}_j \neq 0$. The condition $\mathbf{rD} = 0$ implies

$$P_j(r) \coloneqq \langle \mathbf{r}, \mathbf{d}_j \rangle = \sum_{i=0}^{k-1} (\mathbf{D})_{i,j} r^i = 0.$$

Since P_j is a nonzero polynomial of degree at most $k-1$ and r is sampled after the input commitments are fixed, Lemma 3.2 gives

$$\Pr_{r \leftarrow \$ \mathbb{F}}[\mathbf{rD} = 0] \le \Pr_{r \leftarrow \$ \mathbb{F}}[P_j(r) = 0] \le \frac{k-1}{|\mathbb{F}|}.$$

Proximity sampling. Now condition further on $\mathbf{rD} \neq 0$. No vectors can simultaneously satisfy all four global relations:

$$\mathbf{x} = \mathbf{rA}, \quad \mathbf{y} = \mathbf{xB}, \quad \mathbf{z} = \mathbf{rC}, \quad \mathbf{y} = \mathbf{z}.$$

Otherwise the four equalities would imply $\mathbf{rAB} = \mathbf{rC}$, contradicting $\mathbf{rD} \neq 0$. Thus at least one relation is false. Consider $\mathbf{x} \neq \mathbf{rA}$; the other cases are identical. Injectivity and distance of the code imply that the two codewords $\mathsf{Enc}(\mathbf{x})$ and $\mathsf{Enc}(\mathbf{rA})$ disagree on at least δn coordinates. By code linearity, the sampled fold value from the committed rows of $\mathbf{A}$ equals $\mathsf{Enc}(\mathbf{rA})_j$ at coordinate j. Since the sampled column used by Groth16 is bound to the authenticated input leaf handle, and the sampled vector value is both bound to its authenticated leaf handle and checked to equal the circuit-computed codeword $\mathsf{Enc}(\mathbf{x})_j$, the fold check rejects whenever j is in the disagreement set. A single uniformly sampled coordinate misses this disagreement set with probability at most $1 - \delta$, and t independent sampled coordinates all miss it with probability at most $(1 - \delta)^t$. Union-bounding over the four possible violated relations yields $4(1 - \delta)^t$.

Combining the bounds and parameters. Summing the backend, Freivalds, and proximity errors gives

$$\Pr[\mathsf{Acc} \mid \mathbf{AB} \neq \mathbf{C}] \leq \epsilon_{\mathsf{SNARK}}(\lambda) + \epsilon_{\mathsf{cc}}(\lambda) + \epsilon_{\mathsf{bind}}(\lambda) + \frac{k-1}{|\mathbb{F}|} + 4(1-\delta)^t.$$

For $\lambda = 128$, a 256-bit field makes $(k-1)/|\mathbb{F}|$ negligible for the reported dimensions. The proximity term is at most $2^{-\lambda}$ when $t = \lceil (\lambda + 2)/\log_2(1/(1-\delta)) \rceil$; for example, if the backend certifies $\delta = 1/2$, then $t = 130$ suffices for the proximity term alone. For a conservative aggregate 128-bit budget over all terms, one should allocate the budget across terms, e.g., by targeting each term at 2^{-131}.

Fiat–Shamir variant. The non-interactive transcript derives both r and I from domain-separated hashes of the transcript prefix, including the public input roots, intermediate message commitments, and encoded-vector roots. In the random oracle model, the preceding public-coin proof applies with the standard Fiat–Shamir loss $\epsilon_{\mathsf{FS}}(q_H)$ for an adversary that makes q_H random-oracle queries. This proves Theorem 5.1. □

Appendix B. CP-Link Backend Instantiation

The artifact uses Pedersen-style [29] handles

$$\Gamma = \rho_\Gamma H + \sum_{\ell=1}^{d_\chi} a_\ell G_\ell, \qquad L = \rho_L H + \sum_{\ell=1}^{d_\chi} a_\ell G_\ell,$$

with $d_\chi = k$ for matrix columns and $d_\chi = 1$ for vector coordinates. Leaf handles are authenticated by the Merkle tree, while Groth16 checks the wire-handle opening for the sampled witness value. The link proof shows that the public wire handle and the authenticated leaf handle open to the same sampled object.

Theorem B.1 (Batched CP-link backend)**.** *Assume the Pedersen handle parameters are generated so that the handle commitment is binding, the QA-NIZK link proof for the linear batch relation is sound, and the batching challenge is sampled by the Fiat–Shamir random oracle after the batch statement is fixed. For $Q = |\mathcal{S}|t$ sampled objects, the batched CP-link verifier accepts a false batch with probability at most $\epsilon_{\mathsf{ped}} + \epsilon_{\mathsf{QA}} + Q/|\mathbb{F}|$. The artifact serializes the Q logical relations as one online batched proof; generator and CRS bytes are setup material and are not included in the online proof-size claim.*

The logical CP-link relation contains one equation per sampled object:

$$\Gamma_q - L_q = \Delta_q H, \qquad q \in [Q],$$

where $Q = |\mathcal{S}|t$ and q indexes the pairs $(\chi, j) \in \mathcal{S} \times I$. The batch statement is

$$P_I = \{(\chi_q, j_q, \Gamma_q, L_q)\}_{q=1}^{Q},$$
$$T_I = \mathsf{Hash}(\mathsf{FMM\text{-}link\text{-}batch}, \mathsf{Cm}_{ABC}, \mathsf{Cm}_{XYZ}, I, P_I).$$

The implementation batches these equations by deriving $\beta \leftarrow \mathsf{Hash}(\mathsf{FMM\text{-}link\text{-}rand}, T_I, \{\Gamma_q, L_q\}_{q=1}^{Q})$ and checking a single randomized linear relation

$$\sum_{q=1}^{Q} \beta^q (\Gamma_q - L_q) = \left(\sum_{q=1}^{Q} \beta^q \Delta_q \right) H.$$

If any individual equality is false, the left-minus-right side is a nonzero degree-Q polynomial in β, so the batch can pass with probability at most $Q/|\mathbb{F}|$ in the random-oracle model. Combining this with Pedersen binding and QA-NIZK soundness gives the bound in Theorem B.1. The QA-NIZK statement is $(\mathsf{pp}, T_I, \{\Gamma_q, L_q\}_{q=1}^{Q})$ and the witness is the aggregate difference opening $\sum_q \beta^q \Delta_q$. Its CRS fixes the handle generators and linear-relation verification parameters; if serialized, those bytes are setup/key/CRS material rather than online proof bytes. Thus the 128-byte value reported in the experiments is one online batched QA-NIZK proof for all sampled links, not a per-link cost.

Appendix C. Additional Evaluation Details

This appendix records the measurement boundary behind the main evaluation. Tables 6 and 7 first state the fixed experimental parameters and which costs are covered by the artifact. Tables 8–11 then separate serialized proof material from private Groth16 witness data and from costs that remain unmeasured, such as raw input certification and serialized key/CRS size. The intent is to make the main text's proof-size and speedup claims auditable without treating the checking-layer measurements as full end-to-end VC costs.

Table 6 fixes the common parameter choices used by the reported rows. In particular, the measured rows use $t = 128$ for scaling, while the table also records the nearby query counts needed for stricter proximity-only and per-term security budgets.

Parameter	Value in main experiments
Field size	256-bit prime field
Server	AMD EPYC 7B13, 32 cores, 251.69 GiB RAM, Linux/amd64
Matrix dimensions	$\mathsf{FastMatMul}_{\mathsf{QA}}$: 2^7–2^{13}; Freivalds: 2^7–2^{12}
Code rate	$\rho = 1/2$
Codeword length	$n = 2k$
Sampled positions	$t = 128$
Primary linking layer	Message commitments, wire handles, QA-NIZK/CP-link, and multi-column Merkle openings
Backend model	CP-linked sampled intermediate backend; certified input roots are assumed
Query-count targets	Measured rows use $t = 128$; proximity-only 2^{-128} at $\delta = 1/2$ uses $t = 130$; conservative per-term 2^{-131} uses $t = 133$.

TABLE 6. CONCRETE PARAMETERS USED IN THE MAIN EXPERIMENTS.

Table 7 is a measurement-scope checklist rather than a performance result. It distinguishes quantities measured directly by the CSV artifact from costs that remain deployment assumptions, most notably raw input certification and serialized key/CRS sizes.

Metric	Covered?	Interpretation
Constraint count	Yes, aggregate	Groth16 circuit size including message openings, wire-handle openings, sampled RS evaluations, and fold checks; per-gadget constraint counters are not reported, and auxiliary CP-link verification is measured in time/proof bytes rather than constraints.
Online proving time	Yes	Excludes native matrix multiplication and one-time setup.
Verification time	Yes	Includes current QA-NIZK/CP-link and opening verification overhead.
Proof size	Yes	Includes Groth16, compact sampled handles/openings, and QA-NIZK material; raw sampled matrix columns are Groth16 witness data, not serialized proof bytes.
Setup time	Partial	Recorded for the matrix and GPT-2 workloads; setup/key sizes are not reported.
Raw input certification	No	Required to close the full raw-input-to-certified-commitment pipeline.
Peak memory and variance	No	Needed for a full systems artifact evaluation.

TABLE 7. MEASUREMENT COVERAGE IN THE CURRENT ARTIFACT.

Table 8 explains the proof-size convention used in the main text. The sampled matrix columns are large, but they are private Groth16 witness data; the serialized proof contains only compact handles, authentication paths, the Groth16 proof, and the batched CP-link material.

Object	Serialized?	Where it is checked / charged
Input column $a_{M,j}$	No	k field elements are Groth16 witness data; handles and paths are serialized.
Vector leaf $a_{v,j}$	No	One field element is checked by Groth16 via $\mathsf{Wire.Verify}$ and $a_{v,j} = \mathsf{Enc}(v)_j$.
Merkle path	Yes	Authenticates $L_{\chi,j}$ in AuxV; charged to proof bytes and verify time.
CP-link proof	Yes	One batched QA-NIZK proof τ_I links all sampled handle pairs.
Groth16 proof	Yes	Verifies message openings, wire openings, sampled RS evaluations, and fold/equality checks.

TABLE 8. PROOF-MATERIAL ACCOUNTING FOR THE CP-LINKED SAMPLED-OPENING BACKEND.

Table 9 instantiates the accounting for two representative matrix sizes. The reported totals are the serialized online proof bytes in the artifact; roots and message commitments are public inputs, and raw sampled columns are intentionally not serialized.

Case	Opening payload	Online proof	Note
$k = 1{,}024$	108,160 B	108,548 B	Adds 260 B Groth16 and 128 B batched CP-link; roots/messages are public inputs.
$k = 4{,}096$	146,400 B	146,788 B	Raw sampled columns would be 50.3 MB if serialized.

TABLE 9. REPRESENTATIVE BYTE-LEVEL ACCOUNTING FROM `MEOW_BENCHMARK_RESULTS.CSV`.

Table 10 records setup time for representative rows and separates protocol, circuit, and CP-link setup components. The current artifact does not report serialized proving/verifying-key or CRS bytes; this is why the main text states proof size as online proof material excluding key/CRS data.

Case	Total setup	Protocol	Circuit	CP-link	Key / CRS size
$k = 1{,}024$	32.99 s	0.06 s	31.81 s	1.12 s	Not reported by the current artifact.
$k = 4{,}096$	128.21 s	0.23 s	123.64 s	4.34 s	Not reported by the current artifact.

TABLE 10. SETUP ACCOUNTING FOR REPRESENTATIVE ROWS.

Table 11 separates the dominant deterministic $3tk$ fold terms from the remaining checked constraints. The residual includes sampled RS evaluations, wire/message openings, equality checks, and circuit glue, so the table is a structural audit of the aggregate constraint counts rather than a per-gadget profiler.

Case	Total	$3tk$ folds	Residual	Raw cols.
$k = 1{,}024$	419,139	393,216	25,923	12.6 MB
$k = 4{,}096$	1,667,305	1,572,864	94,441	50.3 MB

TABLE 11. STRUCTURAL CONSTRAINT AND RAW-COLUMN-SIZE ACCOUNTING FOR TWO REPRESENTATIVE ROWS.

The accounting tables should be read as lower-level support for the main evaluation, not as additional benchmark claims. They justify three statements used throughout the paper: the large sampled columns remain private witness data, the serialized online proof is dominated by compact opening material rather than raw columns, and the current artifact still leaves key/CRS size and raw-input certification as unmeasured deployment costs.